

Конспект к экзамену

Алгоритмы и структуры данных

Автор: Евграфов Артём

18 июня 2026 г.

Содержание

1	Алгоритм, анализ сложности алгоритма. Асимптотическая оценка сложности алгоритма. Классы функций O, o, Θ, Ω.	1
1.1	Сложность алгоритма	1
1.2	Асимптотические обозначения	1
1.3	Определения через пределы	1
1.4	Амортизированный анализ	2
1.5	Примеры	3
1.6	Основные классы сложности	4
1.7	Мастер-теорема	4
1.8	Экспоненциальные рекуррентные соотношения	5
2	Задача сортировки. Обзор и сравнительная характеристика методов сортировки.	7
2.1	Постановка задачи сортировки	7
2.2	Сортировки, основанные на сравнениях	7
2.3	Свойства методов сортировки	7
2.4	Сравнительная характеристика алгоритмов	8
2.5	Краткий обзор методов сортировки	8
2.6	Оценка снизу на время сортировки	9
3	Задача сортировки. Квадратичные сортировки: выбором, пузырьком, вставками. Сложность и корректность алгоритмов.	10
3.1	Основные определения	10
3.2	Сортировка выбором	10
3.3	Сортировка вставками	11
3.4	Сортировка пузырьком	11
4	Задача сортировки. Сортировка слиянием. Сложность и корректность сортировки.	12
4.1	Идея алгоритма	12
4.2	Алгоритм MergeSort	13
4.3	Сложность	13
5	Задача сортировки. Сортировка пирамидой (кучей). Сложность и корректность сортировки.	14
5.1	Бинарная куча	14
5.2	Операция siftDown	15

5.3	Построение кучи	15
5.4	Алгоритм HeapSort	16
5.5	Сложность и свойства	17
5.6	Pairing heap	17
6	Задача сортировки. QuickSort. Сложность и корректность сортировки.	18
6.1	Идея алгоритма	18
6.2	Разбиение массива	18
6.3	Корректность QuickSort	19
6.4	Сложность	19
6.5	Память и свойства	20
7	Структуры данных. Массив, стек, очередь, одно- и двусвязные списки.	21
7.1	Понятие структуры данных	21
7.2	Массив	21
7.3	Расширяющийся массив	21
7.4	Односвязный список	22
7.5	Двусвязный список	23
7.6	Стек	23
7.7	Очередь	24
7.8	Дек	24
8	Бинарные деревья поиска. Обход дерева. Операции вставки, удаления.	25
8.1	Определение BST	25
8.2	Поиск	25
8.3	Обходы дерева	25
8.4	Минимум, максимум и следующий элемент	26
8.5	Вставка	26
8.6	Удаление	27
8.7	Lower bound и upper bound	28
8.8	Высота дерева и балансировка	28
9	Задача балансировки бинарных деревьев. Красно-чёрные деревья: определение, вычислительная сложность операций поиска, вставки и удаления.	29
9.1	Зачем нужна балансировка	29
9.2	Определение красно-чёрного дерева	30
9.3	Оценка высоты	30

9.4	Поиск	31
9.5	Вставка	31
9.6	Удаление	31
9.7	Итоговые оценки	32
10	Задача хеширования. Хеш-функции. Таблица с прямой адресацией.	32
10.1	Задача словаря	32
10.2	Прямая адресация	33
10.3	Хеш-функция	34
10.4	Коэффициент заполнения и rehash	34
11	Разрешение коллизий при хешировании. Открытая адресация.	35
11.1	Идея открытой адресации	35
11.2	Состояния ячеек	35
11.3	Сложность линейного пробирования	36
11.4	Виды пробирования	36
11.5	Удаления и накопление могил	37
12	Разрешение коллизий в хеш-таблицах на основе списков.	37
12.1	Метод цепочек	37
12.2	Оценка времени	38
12.3	Перевыделение таблицы	38
12.4	Сравнение с открытой адресацией	39
12.5	Практические замечания	39
13	Графы. Ориентированные и неориентированные графы, деревья. Пути, циклы. Задача связности. Представление графов. Лемма о рукопожатиях.	39
13.1	Основные определения	39
13.2	Связность	40
13.3	Представление графов	41
14	Графы. Обход графа в ширину. Корректность и сложность алгоритма. Структура дерева поиска. Примеры использования алгоритма.	41
14.1	Идея BFS	41
14.2	Сложность	42
14.3	Применения BFS	42
15	Графы. Обход графа в глубину. Корректность и сложность алгоритма.	

Структура дерева поиска. Примеры использования алгоритма.	42
15.1 Идея DFS	42
15.2 Сложность	43
15.3 Времена входа и выхода	43
15.4 Применения DFS	44
16 Графы. Топологическая сортировка. Корректность и сложность алгоритма. Поиск циклов в графе.	44
16.1 Определение	44
16.2 Алгоритм Кана	44
16.3 Топологическая сортировка через DFS	45
16.4 Поиск циклов	45
17 Графы. Поиск сильно связанных компонент. Корректность и сложность алгоритма.	46
17.1 Компоненты сильной связности	46
17.2 Алгоритм Косарайю	46
17.3 Сложность и применения	47
18 Графы. Задача поиска кратчайшего расстояния в графе. Алгоритм Дейкстры. Корректность и сложность алгоритма.	48
18.1 Постановка задачи	48
18.2 Алгоритм Дейкстры	48
18.3 Сложность	49
18.4 Восстановление пути	50
18.5 Ограничения и замечания	50
19 Графы. Минимальные остовные деревья. Алгоритм Прима. Корректность и сложность алгоритма.	50
19.1 Минимальное остовное дерево	50
19.2 Алгоритм Прима	51
19.3 Сложность	52
19.4 Алгоритм Краскала	52
20 Графы. Задача поиска кратчайших расстояний в графе. Алгоритм Беллмана–Форда. Поиск циклов отрицательного веса.	53
20.1 Задача и отличие от Дейкстры	53
20.2 Динамическая идея	53
20.3 Алгоритм	53

20.4 Сложность	54
20.5 Поиск отрицательного цикла	54
20.6 Восстановление отрицательного цикла	55
20.7 Практические оптимизации	55

1 Алгоритм, анализ сложности алгоритма. Асимптотическая оценка сложности алгоритма. Классы функций O , o , Θ , Ω .

1.1 Сложность алгоритма

Время работы алгоритма рассматривается как функция от размера входных данных. Размер входа обозначается через n .

Время работы

Функция $f(n)$ — время работы алгоритма в зависимости от параметра задачи n , который обычно является размером входных данных.

Далее предполагаем, что

$$n \in \mathbb{N}, \quad f(n) > 0.$$

Асимптотический анализ изучает поведение функции $f(n)$ при $n \rightarrow +\infty$.

1.2 Асимптотические обозначения

Рассмотрим функции

$$f, g : \mathbb{N} \rightarrow \mathbb{R}^{>0}.$$

Классы O , Ω , Θ , o , ω

- $f = O(g)$, если

$$\exists N > 0, \exists C > 0 : \forall n \geq N, \quad f(n) \leq Cg(n).$$

- $f = \Omega(g)$, если

$$\exists N > 0, \exists C > 0 : \forall n \geq N, \quad f(n) \geq Cg(n).$$

- $f = \Theta(g)$, если

$$\exists N > 0, \exists C_1 > 0, \exists C_2 > 0 : \forall n \geq N, \quad C_1 g(n) \leq f(n) \leq C_2 g(n).$$

- $f = o(g)$, если

$$\forall C > 0 \exists N > 0 : \forall n \geq N, \quad f(n) \leq Cg(n).$$

- $f = \omega(g)$, если

$$\forall C > 0 \exists N > 0 : \forall n \geq N, \quad f(n) \geq Cg(n).$$

1.3 Определения через пределы

Асимптотические обозначения можно описывать через отношение

$$\frac{f(n)}{g(n)}.$$

Через пределы

- $f = o(g)$, если

$$\lim_{n \rightarrow +\infty} \frac{f(n)}{g(n)} = 0.$$

- $f = O(g)$, если

$$\overline{\lim}_{n \rightarrow +\infty} \frac{f(n)}{g(n)} < +\infty.$$

- $f = \Omega(g)$, если

$$\underline{\lim}_{n \rightarrow +\infty} \frac{f(n)}{g(n)} > 0.$$

- $f = \Theta(g)$, если

$$0 < \underline{\lim}_{n \rightarrow +\infty} \frac{f(n)}{g(n)} \leq \overline{\lim}_{n \rightarrow +\infty} \frac{f(n)}{g(n)} < +\infty.$$

- $f = \omega(g)$, если

$$\lim_{n \rightarrow +\infty} \frac{f(n)}{g(n)} = +\infty.$$

Замечание. Обычный предел отношения $\frac{f(n)}{g(n)}$ может не существовать. При этом классы O , Ω и Θ остаются определёнными.

Верхний и нижний пределы существуют всегда в расширенной вещественной прямой:

$$\underline{\lim}_{n \rightarrow +\infty} \frac{f(n)}{g(n)} \quad \text{и} \quad \overline{\lim}_{n \rightarrow +\infty} \frac{f(n)}{g(n)}.$$

Они могут принимать значения $+\infty$ и $-\infty$. В нашем случае $f(n) > 0$ и $g(n) > 0$, поэтому отношение положительно, а возможные значения лежат в диапазоне от 0 до $+\infty$.

1.4 Амортизированный анализ

В худшем случае отдельная операция структуры данных может быть дорогой, но такие дорогие операции могут происходить редко. Амортизированный анализ оценивает не одну операцию, а произвольную последовательность операций.

Амортизированное время

Пусть выполнена последовательность из m операций, реальные времена которых равны $t_1, t_2, \dots, t_m$. Если для любой такой последовательности

$$\sum_{i=1}^m t_i = O(mA),$$

то говорят, что амортизированное время одной операции равно $O(A)$.

Амортизированная оценка не является средней по вероятности: она не предполагает случайного распределения входов и даёт гарантию для любой последовательности операций.

Методы амортизированного анализа

- **Метод агрегирования:** напрямую оценивается суммарная стоимость всей последовательности операций, затем сумма делится на число операций.
- **Бухгалтерский метод:** каждой операции заранее назначается амортизированная стоимость. Дешёвые операции могут оставлять «запас», который затем оплачивает дорогие операции.
- **Метод потенциалов:** состоянию структуры данных сопоставляется потенциал $\Phi_i \geq 0$. Амортизированная стоимость i -й операции определяется как

$$a_i = t_i + \Phi_i - \Phi_{i-1}.$$

В методе потенциалов суммы телескопируются:

$$\sum_{i=1}^m a_i = \sum_{i=1}^m t_i + \Phi_m - \Phi_0.$$

Если $\Phi_0 = 0$ и $\Phi_i \geq 0$, то

$$\sum_{i=1}^m t_i \leq \sum_{i=1}^m a_i.$$

Поэтому достаточно ограничить суммарную амортизированную стоимость.

Стандартные примеры

- В расширяющемся массиве `push_back` иногда вызывает перевыделение и копирование $O(n)$ элементов, но при удвоении размера суммарное число копирований за m вставок равно $O(m)$. Значит `push_back` работает за $O(1)$ амортизированно.
- В стеке операция `pop(k)`, удаляющая до k элементов, может стоить $O(k)$. Однако каждый элемент может быть удалён не более одного раза, поэтому любая последовательность из m операций `push` и `pop` имеет суммарную стоимость $O(m)$, если считать элементарной операцией добавление или удаление одного элемента.

1.5 Примеры

Базовые примеры

- Для любого фиксированного $k \geq 0$:

$$n^k = o(n^{k+1}).$$

- Если $P(x)$ — многочлен со старшим коэффициентом > 0 , то

$$P(x) = \Theta(x^{\deg P}).$$

1.6 Основные классы сложности

Типовые классы

- Линейная сложность:

$$O(n).$$

- Квадратичная сложность:

$$O(n^2).$$

- Полиномиальная сложность:

$$\exists k > 0 : O(n^k).$$

- Полилогарифмическая сложность:

$$\exists k > 0 : O(\log^k n).$$

- Экспоненциальная сложность:

$$\exists c > 0 : O(2^{cn}).$$

1.7 Мастер-теорема

Мастер-теорема для простой рекуррентности

Пусть

$$T(n) = aT\left(\frac{n}{b}\right) + f(n), \quad f(n) = n^c,$$

где $a > 0$, $b > 1$, $c \geq 0$. Обозначим глубину рекурсии:

$$k = \log_b n.$$

Тогда:

$$\begin{cases} T(n) = \Theta(a^k) = \Theta(n^{\log_b a}), & a > b^c, \\ T(n) = \Theta(f(n)) = \Theta(n^c), & a < b^c, \\ T(n) = \Theta(kf(n)) = \Theta(n^c \log n), & a = b^c. \end{cases}$$

Доказательство. Раскроем рекуррентность:

$$\begin{aligned} T(n) &= f(n) + aT\left(\frac{n}{b}\right) \\ &= f(n) + af\left(\frac{n}{b}\right) + a^2f\left(\frac{n}{b^2}\right) + \dots \\ &= n^c + a\left(\frac{n}{b}\right)^c + a^2\left(\frac{n}{b^2}\right)^c + \dots + a^k\left(\frac{n}{b^k}\right)^c. \end{aligned}$$

Вынесем $f(n) = n^c$:

$$T(n) = f(n) \left(1 + \frac{a}{b^c} + \left(\frac{a}{b^c}\right)^2 + \dots + \left(\frac{a}{b^c}\right)^k \right).$$

Обозначим

$$q = \frac{a}{b^c}, \quad S(q) = 1 + q + q^2 + \dots + q^k.$$

Тогда

$$T(n) = f(n)S(q).$$

Рассмотрим три случая. Если $q = 1$, то

$$S(q) = k + 1 = \log_b n + 1 = \Theta(\log n).$$

Значит,

$$T(n) = \Theta(f(n) \log n) = \Theta(n^c \log n).$$

Если $q < 1$, то

$$S(q) = \frac{1 - q^{k+1}}{1 - q} = \Theta(1).$$

Значит,

$$T(n) = \Theta(f(n)) = \Theta(n^c).$$

Если $q > 1$, то

$$S(q) = \frac{q^{k+1} - 1}{q - 1} = \Theta(q^k).$$

Тогда

$$T(n) = \Theta(f(n)q^k) = \Theta\left(n^c \left(\frac{a}{b^c}\right)^k\right) = \Theta\left(a^k \left(\frac{n}{b^k}\right)^c\right).$$

Так как $k = \log_b n$, то $b^k = n$, поэтому

$$\left(\frac{n}{b^k}\right)^c = 1.$$

Следовательно,

$$T(n) = \Theta(a^k) = \Theta(n^{\log_b a}).$$

□

1.8 Экспоненциальные рекуррентные соотношения

Экспоненциальная рекуррентность

Пусть

$$T(n) = \sum_i b_i T(n - a_i),$$

где

$$a_i > 0, \quad b_i > 0, \quad \sum_i b_i > 1.$$

Тогда

$$T(n) = \Theta(\alpha^n),$$

где $\alpha > 1$ — единственный корень уравнения

$$1 = \sum_i b_i \alpha^{-a_i}.$$

Этот корень можно найти бинарным поиском.

Доказательство. Предположим, что решение имеет вид

$$T(n) = \alpha^n.$$

Тогда

$$\alpha^n = \sum_i b_i \alpha^{n-a_i}.$$

Делим обе части на α^n :

$$1 = \sum_i b_i \alpha^{-a_i}.$$

Обозначим

$$h(\alpha) = \sum_i b_i \alpha^{-a_i}.$$

Нужно решить уравнение

$$h(\alpha) = 1$$

при $\alpha \in [1, +\infty)$. При $\alpha = 1$:

$$h(1) = \sum_i b_i > 1.$$

При $\alpha \rightarrow +\infty$:

$$h(\alpha) \rightarrow 0 < 1.$$

Кроме того, $h(\alpha)$ строго убывает на $[1, +\infty)$, так как $a_i > 0$ и $b_i > 0$. Поэтому уравнение $h(\alpha) = 1$ имеет единственный корень $\alpha > 1$. Его можно найти бинарным поиском. Докажем верхнюю оценку $T(n) = O(\alpha^n)$. Выберем такую константу $C > 0$, что

$$T(n) \leq C\alpha^n$$

для всех базовых значений

$$n \in \left[1 - \max_i a_i, 1\right].$$

Это база индукции. Пусть для меньших аргументов уже доказано:

$$T(n - a_i) \leq C\alpha^{n-a_i}.$$

Тогда

$$T(n) = \sum_i b_i T(n - a_i) \leq \sum_i b_i C\alpha^{n-a_i} = C\alpha^n \sum_i b_i \alpha^{-a_i} = C\alpha^n.$$

Последнее равенство верно, потому что α является корнем уравнения

$$1 = \sum_i b_i \alpha^{-a_i}.$$

Значит,

$$T(n) = O(\alpha^n).$$

Нижняя оценка доказывается аналогично. Выберем $c > 0$ так, что

$$T(n) \geq c\alpha^n$$

на базовом отрезке. Тогда по индукции:

$$T(n) = \sum_i b_i T(n - a_i) \geq \sum_i b_i c\alpha^{n-a_i} = c\alpha^n \sum_i b_i \alpha^{-a_i} = c\alpha^n.$$

Следовательно,

$$T(n) = \Omega(\alpha^n).$$

Так как одновременно выполнены оценки

$$T(n) = O(\alpha^n) \quad \text{и} \quad T(n) = \Omega(\alpha^n),$$

получаем

$$T(n) = \Theta(\alpha^n).$$

□

2 Задача сортировки. Обзор и сравнительная характеристика методов сортировки.

2.1 Постановка задачи сортировки

Пусть дана последовательность элементов

$$a_1, a_2, \dots, a_n.$$

Задача сортировки состоит в том, чтобы переставить элементы последовательности так, чтобы получилась неубывающая последовательность:

$$a_1 \leq a_2 \leq \dots \leq a_n.$$

Если каждый элемент имеет ключ, то сортировка выполняется по значению ключа.

Сортировка

Сортировка — это задача упорядочивания элементов заданной последовательности по некоторому отношению порядка. Результатом сортировки является перестановка исходных элементов, расположенная в требуемом порядке.

2.2 Сортировки, основанные на сравнениях

Если при сортировке объектов разрешено обращаться с этими объектами единственным способом — сравнивать их на больше/меньше, то такая сортировка называется сортировкой, основанной на сравнениях.

В этой модели алгоритм получает информацию о входных данных только из результатов сравнений вида

$$a_i < a_j, \quad a_i > a_j, \quad a_i = a_j.$$

2.3 Свойства методов сортировки

Методы сортировки обычно сравнивают по следующим характеристикам:

- асимптотическая сложность в лучшем, среднем и худшем случае;
- объём дополнительной памяти;
- устойчивость;
- применимость к внутренней или внешней сортировке;
- использование только сравнений или дополнительных ограничений на данные.

Устойчивость сортировки

Сортировка называется устойчивой, если равные элементы сохраняют свой относительный порядок после сортировки.

Адаптивность сортировки

Сортировка называется адаптивной, если её время работы улучшается на почти отсортированных массивах. Например, сортировка вставками работает за $O(n + I)$, где I — число инверсий, поэтому на почти отсортированных данных она близка к линейной.

Внутренняя и внешняя сортировка

Внутренняя сортировка предполагает, что все данные помещаются в оперативную память. Внешняя сортировка применяется, когда данные слишком велики и хранятся во внешней памяти.

2.4 Сравнительная характеристика алгоритмов

Алгоритм	Лучший случай	Средний случай	Худший случай	Память	Устойч.
Пузырьком	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	да
Вставками	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	да
Выбором	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	нет
Слиянием	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	да
Быстрая	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	нет
Пирамидальная	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	нет
Подсчётом	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(n + k)$	да

Здесь k — размер диапазона возможных значений ключей для сортировки подсчётом. Лучший случай $O(n)$ для сортировки пузырьком указан для варианта с ранней остановкой, если на очередном проходе не было обменов. Оценка памяти для QuickSort указана для реализации, которая рекурсивно обрабатывает меньшую часть, а большую часть продолжает сортировать итеративно.

2.5 Краткий обзор методов сортировки

Простые квадратичные сортировки

- Сортировка пузырьком последовательно сравнивает соседние элементы и меняет их местами, если они стоят в неправильном порядке.
- Сортировка вставками поддерживает отсортированный префикс и вставляет очередной элемент в нужное место.
- Сортировка выбором на каждом шаге выбирает минимальный элемент из неотсортированной части и переносит его в начало этой части.

Эти алгоритмы имеют сложность $O(n^2)$ в среднем и худшем случае.

Эффективные сортировки сравнениями

- Сортировка слиянием рекурсивно делит массив на две части, сортирует их и затем сливает два отсортированных массива.
- Быстрая сортировка выбирает опорный элемент, разбивает массив на элементы меньше и больше опорного, затем рекурсивно сортирует части.
- Пирамидальная сортировка строит двоичную кучу и последовательно извлекает из неё максимум или минимум.

Эти алгоритмы имеют порядок роста $O(n \log n)$ в типичных или гарантированных случаях, за исключением худшего случая быстрой сортировки.

Линейные сортировки

Сортировка подсчётом, поразрядная сортировка и блочная сортировка могут работать быстрее $O(n \log n)$, но они не являются произвольными сортировками сравнениями. Для них нужны дополнительные ограничения на входные данные, например ограниченный диапазон ключей.

2.6 Оценка снизу на время сортировки

Нижняя оценка для сортировки сравнениями

Любая корректная детерминированная сортировка, основанная на сравнениях, делает в худшем случае хотя бы

$$\Omega(n \log n)$$

сравнений.

Доказательство. Докажем, что существует тест вида «перестановка», на котором сортировка должна сделать $\Omega(n \log n)$ сравнений. Всего существует $n!$ различных перестановок. Пусть сортировка делает не более k сравнений. Можно считать, что она делает ровно k сравнений, добавив при необходимости бесполезные сравнения. Результат каждого сравнения — один бит информации: либо «<», либо «>». Поэтому после k сравнений существует не более 2^k различных наборов ответов.

Если $2^k < n!$, то двум различным перестановкам соответствует один и тот же набор ответов на сравнения. Тогда алгоритм выполнит на них одинаковые действия и выдаст один и тот же результат, значит, хотя бы одну из этих перестановок он отсортирует неверно. Следовательно, для корректной сортировки необходимо

$$2^k \geq n!.$$

Отсюда

$$k \geq \log(n!) = \Theta(n \log n).$$

Значит, любая корректная сортировка сравнениями делает в худшем случае

$$\Omega(n \log n)$$

сравнений. □

3 Задача сортировки. Квадратичные сортировки: выбором, пузырьком, вставками. Сложность и корректность алгоритмов.

3.1 Основные определения

Устойчивая сортировка

Сортировка называется устойчивой, если одинаковые элементы остаются в том же относительном порядке, в котором они находились в исходной последовательности.

Инверсия

Инверсией в массиве называется пара индексов $i < j$, такая что

$$a_i > a_j.$$

Через I будем обозначать число инверсий в массиве.

Критерий отсортированности

Массив отсортирован по неубыванию тогда и только тогда, когда

$$I = 0.$$

3.2 Сортировка выбором

На каждом шаге сортировка выбором выбирает минимальный элемент в неотсортированной части массива и ставит его в начало этой части.

```
1 for (int i = 0; i < n; i++):  
2     j = index of min on [i..n)  
3     swap(a[j], a[i])
```

Корректность сортировки выбором

После i выполненных итераций первые i элементов массива являются i минимальными элементами исходного массива и расположены в правильном порядке.

Доказательство. Докажем утверждение по индукции по i . При $i = 0$ отсортированный префикс пуст, поэтому утверждение верно. Пусть после i итераций первые i элементов уже стоят правильно. На следующем шаге алгоритм выбирает минимальный элемент среди оставшихся и ставит его на позицию i . Следовательно, после $i + 1$ итераций первые $i + 1$ элементов также стоят правильно. После n итераций весь массив отсортирован. $\square$

Сложность. На каждой итерации выполняется поиск минимума в оставшейся части массива, поэтому число сравнений равно

$$(n - 1) + (n - 2) + \dots + 1 = \Theta(n^2).$$

Дополнительная память равна $O(1)$. Сортировка выбором в обычной реализации не является устойчивой.

3.3 Сортировка вставками

Сортировка вставками поддерживает отсортированный префикс. На шаге i элемент a_i вставляется в нужное место среди первых i элементов.

```
1 for (int i = 0; i < n; i++)
2     for (int j = i; j > 0 && a[j] < a[j-1]; j--)
3         swap(a[j], a[j-1]);
```

Корректность сортировки вставками

Перед итерацией внешнего цикла с номером i префикс

$$a_0, a_1, \dots, a_{i-1}$$

отсортирован; после этой итерации отсортирован префикс длины $i + 1$.

Доказательство. Докажем по индукции по i . При $i = 0$ пустой префикс отсортирован. Пусть перед итерацией i первые i элементов отсортированы. Внутренний цикл последовательно меняет местами a_i с предыдущим элементом, пока элемент слева больше него. В результате новый элемент попадает в единственную позицию, где порядок в префиксе сохраняется. Следовательно, после итерации отсортирован префикс длины $i + 1$. После завершения алгоритма отсортирован весь массив. $\square$

Сложность. В худшем случае элемент на каждой итерации проходит через весь отсортированный префикс, поэтому

$$T(n) = \Theta(n^2).$$

В лучшем случае массив уже отсортирован, и внутренний цикл сразу завершается, поэтому

$$T(n) = \Theta(n).$$

Дополнительная память равна $O(1)$. Сортировка вставками устойчива, если при сравнении используется строгое условие $a[j] < a[j - 1]$.

Точнее, время сортировки вставками равно $O(n + I)$, где I — число инверсий. Каждый обмен соседних элементов уменьшает число инверсий ровно на 1, а кроме обменов алгоритм делает $O(n)$ проверок завершения внутреннего цикла.

Место вставки можно искать бинарным поиском. Тогда число сравнений становится $O(n \log n)$, но число сдвигов остаётся $O(n^2)$, поэтому асимптотика времени работы остаётся квадратичной. Среди квадратичных сортировка вставками имеет самую малую константу.

3.4 Сортировка пузырьком

Сортировка пузырьком последовательно сравнивает соседние элементы и меняет их местами, если они стоят в неправильном порядке.

```
1 for (int i = 0; i < n; i++)
2     for (int j = 1; j < n; j++)
3         if (a[j-1] > a[j])
4             swap(a[j-1], a[j]);
```

Корректность сортировки пузырьком

После каждой итерации внешнего цикла очередной максимальный элемент из неотсортированной части оказывается на своём окончательном месте.

Доказательство. Во внутреннем цикле больший из двух соседних элементов перемещается вправо. Поэтому максимальный элемент рассматриваемой части массива после последовательности сравнений оказывается в её конце. После первой итерации внешний цикл ставит на место максимальный элемент, после второй — второй максимум и так далее. После n итераций все элементы стоят на своих местах, значит, массив отсортирован. $\square$

Сложность. В данной реализации выполняется $\Theta(n^2)$ сравнений. Дополнительная память равна $O(1)$. Сортировка пузырьком устойчива, если менять элементы местами только при строгом нарушении порядка $a[j-1] > a[j]$.

Возможна модификация: если на очередном проходе не было ни одного обмена, алгоритм можно остановить. Также существует вариант Shaker sort, в котором направление прохода чередуется.

4 Задача сортировки. Сортировка слиянием. Сложность и корректность сортировки.

4.1 Идея алгоритма

Сортировка слиянием относится к алгоритмам типа «разделяй и властвуй». Массив делится на две примерно равные части, каждая часть сортируется рекурсивно, после чего две отсортированные половины сливаются в один отсортированный массив.

Операция слияния

Пусть отрезки

$$a[l, m), \quad a[m, r)$$

уже отсортированы. Операция $\text{Merge}(l, m, r)$ строит отсортированное объединение этих двух отрезков за линейное время

$$\Theta(r - l).$$

Слияние выполняется методом двух указателей: один указатель идёт по левой половине, другой — по правой. На каждом шаге в буфер записывается меньший из двух текущих элементов.

```
1 Merge(l, m, r, a, buffer):
2     i = l, j = m, k = l
3     while i < m && j < r:
4         if a[i] <= a[j]:
5             buffer[k++] = a[i++]
6         else:
7             buffer[k++] = a[j++]
8     while i < m:
9         buffer[k++] = a[i++]
10    while j < r:
11        buffer[k++] = a[j++]
```

```

12     for k = 1..r-1:
13         a[k] = buffer[k]

```

4.2 Алгоритм MergeSort

```

1 MergeSort(l, r, a, buffer): // [l, r)
2     if r - l <= 1:
3         return
4     m = (l + r) / 2
5     MergeSort(l, m, a, buffer)
6     MergeSort(m, r, a, buffer)
7     Merge(l, m, r, a, buffer)

```

Корректность операции слияния

Если массивы $a[l, m)$ и $a[m, r)$ отсортированы, то после выполнения $\text{Merge}(l, m, r)$ отрезок $a[l, r)$ отсортирован и содержит ровно те же элементы, что и до слияния.

Доказательство. В каждый момент времени в буфере уже записан отсортированный префикс результата, состоящий из минимальных элементов среди ещё не обработанных частей двух отрезков. Действительно, следующим в результат должен попасть меньший из текущих элементов левой и правой частей, так как внутри каждой части элементы уже упорядочены. После того как один из отрезков закончился, оставшаяся часть другого отрезка уже отсортирована и дописывается в конец. Следовательно, буфер содержит отсортированное объединение двух исходных отрезков. $\square$

Корректность сортировки слиянием

После выполнения $\text{MergeSort}(0, n, a, \text{buffer})$ массив a отсортирован.

Доказательство. Докажем по индукции по длине отрезка $r - l$. Если $r - l \leq 1$, отрезок уже отсортирован. Пусть утверждение верно для всех отрезков меньшей длины. Алгоритм делит $a[l, r)$ на отрезки $a[l, m)$ и $a[m, r)$. По предположению индукции после рекурсивных вызовов оба отрезка отсортированы. По корректности слияния процедура Merge превращает их в один отсортированный отрезок $a[l, r)$. Значит, утверждение верно для любой длины. $\square$

4.3 Сложность

Пусть $T(n)$ — время работы сортировки слиянием на массиве длины n . Тогда

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n),$$

поскольку выполняются два рекурсивных вызова на половинах массива и одно линейное слияние. По мастер-теореме

$$T(n) = \Theta(n \log n).$$

Дополнительная память в стандартной реализации равна $\Theta(n)$, так как используется буфер для слияния. Глубина рекурсии равна $\Theta(\log n)$.

Свойства MergeSort

Сортировка слиянием работает за $\Theta(n \log n)$ в худшем, среднем и лучшем случае, использует $\Theta(n)$ дополнительной памяти в стандартной реализации и является устойчивой, если при равенстве элементов сначала брать элемент из левой половины.

Нерекурсивная версия. Можно рассматривать рекурсивный алгоритм как обход дерева рекурсии. Сначала сливаются куски длины 1, затем длины 2, затем длины 4 и так далее. Это даёт итеративную версию:

```
1 for len = 1; len < n; len *= 2:
2     for l = 0; l < n; l += 2 * len:
3         m = min(l + len, n)
4         r = min(l + 2 * len, n)
5         Merge(l, m, r, a, buffer)
```

Если после каждого слоя менять местами роли массивов a и $buffer$, можно избежать копирования буфера обратно после каждого отдельного слияния. Это не меняет асимптотику, но уменьшает константу.

Связь с нижней оценкой. Сортировка слиянием является сортировкой сравнениями. Так как для сортировок сравнениями есть нижняя оценка $\Omega(n \log n)$, MergeSort асимптотически оптимален в этой модели.

5 Задача сортировки. Сортировка пирамидой (кучей). Сложность и корректность сортировки.

5.1 Бинарная куча

Бинарная куча хранится в массиве, индексы которого интерпретируются как вершины почти полного бинарного дерева. Для вершины с индексом i дети имеют индексы

$$2i, \quad 2i + 1,$$

а родитель имеет индекс $\lfloor i/2 \rfloor$. В этом разделе для сортировки будем использовать max-heap.

Max-heap

Max-heap — это массив, задающий бинарное дерево, в котором для каждой вершины i выполнено свойство кучи:

$$a[i] \geq a[2i], \quad a[i] \geq a[2i + 1],$$

если соответствующие дети существуют. Следовательно, максимум всей кучи находится в корне.

Высота бинарной кучи

Высота бинарной кучи из n элементов равна $\lceil \log_2 n \rceil$.

Доказательство. В почти полном бинарном дереве на уровне k могут находиться вершины с номерами от 2^k до $2^{k+1} - 1$. Поэтому длина пути от вершины n до корня равна $\lfloor \log_2 n \rfloor$, и высота дерева имеет тот же порядок. $\square$

5.2 Операция siftDown

Процедура siftDown восстанавливает свойство кучи в ситуации, когда оба поддерева вершины i уже являются кучами, но сама вершина i может нарушать порядок с детьми.

```
1 siftDown(i):
2     while true:
3         l = 2 * i
4         r = 2 * i + 1
5         best = i
6         if l <= heapSize && a[l] > a[best]:
7             best = l
8         if r <= heapSize && a[r] > a[best]:
9             best = r
10        if best == i:
11            break
12        swap(a[i], a[best])
13        i = best
```

Корректность siftDown

Если левое и правое поддерева вершины i являются корректными кучами, то после siftDown(i) всё поддерево вершины i является корректной кучей.

Доказательство. На каждом шаге элемент в вершине i сравнивается с максимальным из детей. Если он не меньше обоих детей, свойство кучи в вершине i выполнено, а поддерева были корректны по условию. Если больший ребёнок больше текущего элемента, обмен помещает больший элемент выше, поэтому свойство кучи в старой вершине i восстанавливается. Возможное нарушение переходит только в то поддерево, куда опустился элемент. Процесс заканчивается в листе или в вершине, где свойство кучи выполнено. $\square$

Сложность. За один шаг элемент опускается на один уровень вниз. Поэтому

$$\text{siftDown} = O(\log n).$$

5.3 Построение кучи

Кучу можно построить за линейное время, если вызвать siftDown для всех вершин снизу вверх.

```
1 BuildHeap(a):
2     heapSize = n
3     for i = n / 2; i >= 1; i--:
4         siftDown(i)
```

Корректность BuildHeap

После выполнения BuildHeap массив $a[1..n]$ является корректной бинарной кучей.

Доказательство. Рассмотрим вершины в порядке убывания индекса. Все листья уже являются кучами. Когда вызывается `siftDown(i)`, оба поддерева вершины i уже обработаны, а значит, являются корректными кучами. По корректности `siftDown` после обработки вершины i всё её поддерево становится кучей. В конце обработан корень, следовательно, весь массив является кучей. $\square$

Время построения кучи

Функция `BuildHeap` работает за $\Theta(n)$.

Доказательство. Оценка $O(n \log n)$ получается немедленно, если считать каждый `siftDown` за $O(\log n)$, но она не точна. Большинство вершин расположено близко к листьям, поэтому для них проталкивание короткое.

В полном бинарном дереве вершин высоты h не более $\frac{n}{2^{h+1}}$. Суммарное время построения оценивается так:

$$\sum_{h=0}^{\lfloor \log n \rfloor} \frac{n}{2^{h+1}} \cdot O(h) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right) = O(n).$$

С другой стороны, нужно хотя бы просмотреть элементы массива, поэтому время равно $\Theta(n)$. $\square$

5.4 Алгоритм HeapSort

Пирамидальная сортировка сначала строит `max-heap`, а затем n раз извлекает максимум. Максимум находится в корне, поэтому его можно поставить в конец текущей неотсортированной части массива.

```

1  HeapSort(a):
2      BuildHeap(a)
3      for i = n; i >= 2; i--:
4          swap(a[1], a[i])
5          heapSize--
6          siftDown(1)
```

Корректность HeapSort

После выполнения `HeapSort` массив отсортирован по неубыванию.

Доказательство. После `BuildHeap` максимум находится в корне. На каждой итерации алгоритм меняет корень с последним элементом текущей кучи. Поэтому максимум текущей неотсортированной части оказывается на своей окончательной позиции. Затем размер кучи уменьшается на один, а `siftDown(1)` восстанавливает свойство кучи на оставшейся части массива.

Инвариант внешнего цикла: суффикс $a[i+1..n]$ уже отсортирован и содержит наибольшие элементы массива, а префикс $a[1..i]$ является кучей. Изначально суффикс пуст, а префикс является кучей после построения. Один шаг переносит максимум префикса в начало суффикса и сохраняет инвариант. После завершения цикла весь массив отсортирован. $\square$

5.5 Сложность и свойства

Построение кучи занимает $\Theta(n)$. Затем выполняется $n - 1$ извлечений максимума, каждое из которых требует одного siftDown за $O(\log n)$. Поэтому

$$T(n) = \Theta(n) + O(n \log n) = O(n \log n).$$

Так как пирамидальная сортировка является сортировкой сравнениями, нижняя оценка $\Omega(n \log n)$ применима к худшему случаю. Следовательно,

$$T(n) = \Theta(n \log n).$$

Свойства HeapSort

Пирамидальная сортировка работает за $\Theta(n \log n)$ в худшем случае, использует $O(1)$ дополнительной памяти и является детерминированной. В стандартной реализации она неустойчива.

Практические замечания. HeapSort гарантирует худшее время $\Theta(n \log n)$ и не требует дополнительного массива, поэтому часто используется как запасной вариант в гибридных сортировках. Например, Introsort начинает как QuickSort, но при слишком большой глубине рекурсии переключается на HeapSort, чтобы избежать квадратичного худшего случая.

5.6 Pairing heap

Куча полезна не только для сортировки, но и как приоритетная очередь. Для приоритетной очереди важны операции

insert, getMin, extractMin, decreaseKey, meld.

Pairing heap

Pairing heap — куча, представленная корневым деревом, где в каждой вершине ключ не больше ключей её детей. Главная операция — meld: чтобы слить две кучи, достаточно сравнить их корни и подвесить корень с большим ключом к корню с меньшим ключом.

Благодаря быстрой операции meld вставка нового элемента сводится к слиянию текущей кучи с одноэлементной кучей. Удаление минимума устроено так: корень удаляется, а его дети сливаются попарно слева направо, затем полученные кучи сливаются справа налево.

Сложности Pairing heap

операция	амортизированная сложность
getMin	$O(1)$
meld	$O(1)$
insert	$O(1)$
extractMin	$O(\log n)$
decreaseKey	обычно эффективно; строгая точная оценка сложнее

Pairing heap часто рассматривают как практичную альтернативу фибоначиевой куче: структура проще, константы меньше, а операции слияния и уменьшения ключа хорошо работают на практике. В отличие от бинарной кучи, pairing heap не хранится компактно в массиве, поэтому для HeapSort обычно используют именно бинарную кучу.

6 Задача сортировки. QuickSort. Сложность и корректность сортировки.

6.1 Идея алгоритма

Быстрая сортировка является сортировкой методом разделяй и властвуй. На текущем отрезке выбирается опорный элемент x , после чего элементы переставляются так, чтобы слева оказались элементы, не превосходящие x , а справа — элементы, не меньшие x . Затем две полученные части сортируются рекурсивно.

```
1 QuickSort(l, r, a):
2     if l >= r:
3         return
4     x = a[random(l, r)]
5     Partition(l, r, x, a, i, j)
6     QuickSort(l, j, a)
7     QuickSort(i, r, a)
```

В отличие от MergeSort, QuickSort обычно реализуется на месте: разделение выполняется обменами внутри исходного массива, поэтому дополнительная память уходит в основном на стек рекурсии.

6.2 Разбиение массива

Partition

Пусть задан отрезок массива $a[l..r]$ и опорное значение x . Процедура Partition переставляет элементы отрезка и возвращает индексы i, j такие, что

$$a[l..j] \leq x, \quad a[i..r] \geq x, \quad j < i.$$

Средняя часть $a[j + 1..i - 1]$ не требует дальнейшей сортировки относительно x .

```
1 Partition(l, r, x, a, i, j):
2     i = l
3     j = r
4     while i <= j:
5         while a[i] < x:
6             i++
7         while a[j] > x:
8             j--
9         if i <= j:
10            swap(a[i], a[j])
11            i++
12            j--
```

Корректность Partition

После завершения Partition все элементы в $a[l..j]$ не превосходят x , а все элементы в $a[i..r]$ не меньше x .

Доказательство. Во время работы поддерживается инвариант: все элементы строго левее i , уже обработанные слева, не превосходят x ; все элементы строго правее j , уже обработан-

ные справа, не меньше x . Указатель i пропускает элементы $< x$, указатель j пропускает элементы $> x$. Если после этого $i \leq j$, то $a[i] \geq x$, а $a[j] \leq x$, поэтому обмен ставит оба элемента в подходящие части, после чего оба указателя сдвигаются. Когда $i > j$, необработанных элементов между указателями не осталось, следовательно, инвариант даёт требуемое разбиение. $\square$

Равные элементы. Если в массиве много одинаковых ключей, обычное двухстороннее разбиение может выполнять лишние сравнения и рекурсивные вызовы. Часто используют трёхчастное разбиение:

$$a[l..p-1] < x, \quad a[p..q] = x, \quad a[q+1..r] > x.$$

После этого рекурсивно сортируются только части $< x$ и $> x$. Такая модификация сохраняет ожидаемое $O(n \log n)$ на произвольных данных и работает за $\Theta(n)$, если все элементы равны.

6.3 Корректность QuickSort

Корректность QuickSort

После выполнения $\text{QuickSort}(l, r, a)$ отрезок $a[l..r]$ отсортирован по неубыванию.

Доказательство. Докажем утверждение индукцией по длине отрезка. Для длины 0 или 1 отрезок уже отсортирован. Пусть длина больше 1. После Partition каждый элемент левой части $a[l..j]$ не больше каждого элемента правой части $a[i..r]$, а элементы средней части равны опорному значению или уже стоят между этими двумя группами. По предположению индукции рекурсивные вызовы сортируют левую и правую части. Тогда весь отрезок является конкатенацией отсортированной левой части, средней части и отсортированной правой части, причём порядок между частями также корректен. Значит, $a[l..r]$ отсортирован. $\square$

6.4 Сложность

Процедура Partition за один проход сдвигает указатели только внутрь отрезка, поэтому работает за $\Theta(r - l + 1)$.

Худший случай. Если опорный элемент каждый раз оказывается минимальным или максимальным, то одна из рекурсивных частей имеет размер $n - 1$, а другая пуста:

$$T(n) = T(n - 1) + \Theta(n) = \Theta(n^2).$$

Такой случай возможен при детерминированном выборе опорного элемента на специально подобранном тесте.

Сбалансированный случай. Если каждый раз деление близко к пополам, то

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) = \Theta(n \log n).$$

Ожидаемое число сравнений

Для массива из n различных элементов QuickSort со случайным равномерным выбором опорного элемента выполняет $O(n \log n)$ сравнений в среднем.

Доказательство. Рассмотрим элементы с рангами $i < j$ в отсортированном массиве. Они сравнятся только тогда, когда первым выбранным опорным элементом среди рангов $i, i + 1, \dots, j$ окажется один из двух концов: i или j . Поэтому

$$\Pr[\text{элементы } i \text{ и } j \text{ сравнятся}] = \frac{2}{j - i + 1}.$$

По линейности математического ожидания ожидаемое число сравнений равно

$$\sum_{1 \leq i < j \leq n} \frac{2}{j - i + 1} = 2 \sum_{d=1}^{n-1} \frac{n-d}{d+1} \leq 2n \sum_{d=1}^n \frac{1}{d} = O(n \log n). = O(n \log n).$$

□

Другой способ получить ту же оценку: с вероятностью хотя бы $\frac{1}{2}$ случайный опорный элемент попадает во вторую или третью четверть отсортированного массива. Тогда размеры частей не превосходят $\frac{3}{4}n$, и суммарный размер задач на каждом уровне рекурсии остаётся $O(n)$, а ожидаемая глубина даёт логарифмический множитель.

6.5 Память и свойства

Свойства QuickSort

Randomized QuickSort работает за $O(n \log n)$ в среднем и за $\Theta(n^2)$ в худшем случае. В стандартной реализации сортировка выполняется на месте, но не является устойчивой.

Если делать два рекурсивных вызова напрямую, глубина рекурсии в худшем случае может быть $\Theta(n)$. Чтобы гарантировать $O(\log n)$ дополнительной памяти на стек, рекурсивно вызываются только от меньшей части, а большая часть обрабатывается итеративно:

```
1 QuickSort(l, r, a):
2     while l < r:
3         x = a[random(l, r)]
4         Partition(l, r, x, a, i, j)
5         if j - l < r - i:
6             QuickSort(l, j, a)
7             l = i
8         else:
9             QuickSort(i, r, a)
10            r = j
```

На каждом уровне в стек попадает отрезок размера не более половины текущего, поэтому глубина стека ограничена $O(\log n)$.

Практические замечания. Для массивов с большим числом равных элементов полезно трёхпутевое разбиение на части $< x, = x, > x$: оно не сортирует заново блок элементов, равных опорному. На практике также часто применяют гибрид Introsort: алгоритм начинает как QuickSort, на маленьких отрезках переключается на InsertionSort, а при слишком большой глубине рекурсии — на HeapSort. Поэтому сохраняется высокая средняя скорость QuickSort и появляется гарантия $O(n \log n)$ в худшем случае.

7 Структуры данных. Массив, стек, очередь, одно- и двусвязные списки.

7.1 Понятие структуры данных

Структура данных

Структура данных — это способ организации данных в памяти вместе с набором операций, которые разрешено выполнять над этими данными. При сравнении структур данных обычно оценивают время операций, объём дополнительной памяти, локальность обращений к памяти и удобство модификаций.

Важно различать интерфейс и реализацию. Например, стек задаётся операциями `push` и `pop`, но реализовать его можно массивом, расширяющимся массивом или списком. Один и тот же интерфейс может иметь разные асимптотики и разные константы.

7.2 Массив

Массив

Массив — последовательный участок памяти, состоящий из элементов одного типа. Элемент с индексом i расположен по адресу

$$\text{addr}(a[i]) = \text{addr}(a[0]) + i \cdot \text{sizeof}(T).$$

Из формулы адреса следует главное свойство массива: доступ к произвольному элементу выполняется за $O(1)$. При этом размер обычного массива фиксирован, поэтому вставка в середину или в начало требует сдвига элементов.

Операция	Время
<code>get(i), set(i, x)</code>	$O(1)$
<code>find(x)</code> без дополнительной структуры	$O(n)$
<code>push_back(x)</code> в обычный фиксированный массив	обычно невозможно без перевыделения
<code>insert(i, x), erase(i)</code>	$O(n)$
<code>pop_back()</code>	$O(1)$

Локальность памяти. Массив хорошо работает с кэшем: последовательный проход по массиву обычно существенно быстрее случайного прохода по тем же элементам. Поэтому на практике массив часто быстрее списка даже в задачах, где асимптотика операций выглядит одинаковой.

7.3 Расширяющийся массив

Обычный массив неудобен, если заранее неизвестен размер. Расширяющийся массив хранит два числа: n — текущее число элементов и $capacity$ — число выделенных ячеек. Если при добавлении $n = capacity$, выделяется новый массив большего размера, обычно в два раза больше, и старые элементы копируются туда. Если текущая вместимость равна 0, её сначала увеличивают до 1.

```

1 push_back(x):
2     if n == capacity:
3         b = new array[2 * capacity]
4         copy a[0..n-1] to b
5         a = b
6         capacity = 2 * capacity
7     a[n] = x
8     n++

```

Амортизированное время push_back

В расширяющемся массиве операция push_back работает за $O(1)$ амортизированно.

Доказательство. Рассмотрим момент, когда массив размера m перевыделяется в массив размера $2m$. До следующего перевыделения произойдёт хотя бы m обычных добавлений. Стоимость копирования $O(m)$ можно распределить между этими m операциями, добавив к каждой $O(1)$. Следовательно, средняя стоимость одной операции между двумя перевыделениями равна $O(1)$. $\square$

В расширяющемся массиве сохраняются быстрые get и set, хорошая локальность памяти и быстрый pop_back, но вставка и удаление в середине по-прежнему требуют $O(n)$ сдвигов.

7.4 Односвязный список

Односвязный список

Односвязный список состоит из вершин, каждая из которых хранит значение и указатель на следующую вершину:

$$\text{Node} = \{\text{value}, \text{next}\}.$$

Первый элемент задаётся указателем head. Пустой список обычно обозначается нулевым указателем.

```

1 push_front(head, x):
2     p = new Node
3     p.value = x
4     p.next = head
5     head = p

```

Операция	Время
push_front(x)	$O(1)$
pop_front()	$O(1)$
get(i), set(i, x)	$O(i)$
find(x)	$O(n)$
push_back(x) без tail	$O(n)$
push_back(x) с tail	$O(1)$

Недостаток односвязного списка состоит в том, что из вершины нельзя перейти к предыдущей. Поэтому удаление известной вершины за $O(1)$ обычно невозможно, если нет указателя на предыдущую вершину. Зато вставка после известной вершины выполняется за $O(1)$.

7.5 Двусвязный список

Двусвязный список

В двусвязном списке каждая вершина хранит два указателя:

$$\text{Node} = \{\text{value}, \text{prev}, \text{next}\}.$$

Часто используют фиктивные вершины `head` и `tail`, чтобы одинаково обрабатывать пустой список, вставку в начало и вставку в конец.

```
1 erase(v):
2     v.prev.next = v.next
3     v.next.prev = v.prev
4
5 insert_after(v, x):
6     p = new Node(x)
7     p.prev = v
8     p.next = v.next
9     v.next.prev = p
10    v.next = p
```

Операция	Время
push_front, push_back	$O(1)$
pop_front, pop_back	$O(1)$
erase(Node*)	$O(1)$
insert около известной вершины	$O(1)$
get(i), find(x)	$O(n)$

Двусвязный список тратит больше памяти на указатели, чем односвязный, и хуже использует кэш, чем массив. Его преимущество — дешёвые локальные модификации около уже известных вершин. Именно поэтому итераторы в списке обычно устойчивы к вставкам и удалениям других элементов.

7.6 Стек

Стек

Стек — структура данных с дисциплиной LIFO: последним пришёл, первым вышел. Основные операции:

`push(x), pop(), top()`.

Стек удобно реализуется расширяющимся массивом: вершина стека находится в конце массива. Тогда `push` работает за $O(1)$ амортизированно, а `pop` и `top` — за $O(1)$ в худшем случае. Через список стек также реализуется за $O(1)$ на операцию, если вершину стека хранить в `head`.

Применения. Стек естественно возникает в рекурсии, обходе графа DFS, проверке правильности скобочной последовательности, разборе выражений и монотонных структурах. Важное расширение — стек с минимумом: вместе со стеком значений хранится стек префиксных минимумов, поэтому `getMin` работает за $O(1)$.

7.7 Очередь

Очередь

Очередь — структура данных с дисциплиной FIFO: первым пришёл, первым вышел. Основные операции:

$\text{push}(x)$, $\text{pop}()$, $\text{front}()$.

Очередь можно реализовать двусвязным или односвязным списком с указателями на начало и конец. Тогда добавление в конец и удаление из начала работают за $O(1)$.

Для реализации на массиве используется циклический буфер. Пусть $start$ указывает на первый элемент, а end — на позицию после последнего. Тогда физический индекс элемента с логическим номером i равен

$$(start + i) \bmod capacity.$$

При заполнении буфер можно удваивать, как расширяющийся массив.

```
1 push(x):  
2     a[end] = x  
3     end = (end + 1) mod capacity  
4  
5 pop():  
6     start = (start + 1) mod capacity
```

Очередь на двух стеках. Очередь можно реализовать двумя стеками L и R . Новые элементы кладутся в R . Если нужно удалить элемент, а L пуст, все элементы из R перекладываются в L . Каждый элемент перекладывается не более одного раза, поэтому операции имеют амортизированное время $O(1)$. Такая реализация важна для очереди с минимумом: если оба стека поддерживают минимум, то минимум очереди равен минимуму из минимумов двух стеков.

7.8 Дек

Дек

Дек — двусторонняя очередь. Он поддерживает добавление и удаление с обоих концов:

push_front , push_back , pop_front , pop_back .

Дек естественно реализуется двусвязным списком за $O(1)$ на каждую из четырёх основных операций. Другая реализация — циклический расширяющийся массив; она сохраняет хорошую локальность памяти и также даёт $O(1)$ амортизированно.

8 Бинарные деревья поиска. Обход дерева. Операции вставки, удаления.

8.1 Определение BST

Бинарное дерево поиска

Бинарное дерево поиска, или BST, — бинарное дерево, в каждой вершине которого хранится ключ x_v , причём для любой вершины v :

$$\forall u \in L(v) \quad x_u < x_v, \quad \forall u \in R(v) \quad x_u > x_v,$$

где $L(v)$ и $R(v)$ — левое и правое поддеревья вершины v .

Если равные ключи разрешены, нужно явно выбрать соглашение. Обычно либо хранят пару (x, id) , либо в вершине дополнительно хранят количество одинаковых ключей, либо разрешают равные ключи только в одном из поддеревьев. Без такого соглашения операции поиска и удаления становятся неоднозначными.

8.2 Поиск

Поиск в BST повторяет бинарный поиск, но вместо индексов массива используются переходы по рёбрам дерева.

```
1 find(v, x):
2     while v != null and v.key != x:
3         if x < v.key:
4             v = v.left
5         else:
6             v = v.right
7     return v
```

Если высота дерева равна h , то поиск работает за $O(h)$, потому что на каждом шаге алгоритм спускается на один уровень. В сбалансированном дереве $h = O(\log n)$, но в обычном BST высота может быть $\Theta(n)$, например если ключи вставлялись в уже отсортированном порядке.

8.3 Обходы дерева

Для бинарного дерева стандартны три рекурсивных обхода:

- preorder: вершина, левое поддерево, правое поддерево;
- inorder: левое поддерево, вершина, правое поддерево;
- postorder: левое поддерево, правое поддерево, вершина.

```
1 inorder(v):
2     if v == null:
3         return
4     inorder(v.left)
```

```

5     print(v.key)
6     inorder(v.right)

```

Сортирующее свойство inorder

Обход inorder бинарного дерева поиска выводит ключи в возрастающем порядке.

Доказательство. Докажем по индукции по размеру поддерева. Для пустого дерева утверждение очевидно. Пусть корень имеет ключ x . По определению BST все ключи левого поддерева меньше x , а все ключи правого поддерева больше x . По предположению индукции inorder выводит левое и правое поддерева в отсортированном порядке. Значит, вся последовательность имеет вид

отсортированные ключи $< x$, x , отсортированные ключи $> x$,

то есть тоже отсортирована. □

Все три обхода посещают каждую вершину ровно один раз, поэтому работают за $\Theta(n)$ и используют $O(h)$ памяти на стек рекурсии.

8.4 Минимум, максимум и следующий элемент

Минимальный ключ в непустом BST находится в самой левой вершине, максимальный — в самой правой. Поэтому

$$\min(T), \max(T) = O(h).$$

Следующий элемент после вершины v , то есть минимальный ключ, строго больший x_v , называется $\text{next}(v)$.

```

1  next(v):
2      if v.right != null:
3          v = v.right
4          while v.left != null:
5              v = v.left
6          return v
7      while v.parent != null and v.parent.right == v:
8          v = v.parent
9      return v.parent

```

Если у вершины есть правое поддерево, следующий элемент — минимум в правом поддерево. Если правого поддерева нет, нужно подниматься вверх, пока текущая вершина является правым сыном; первый предок, для которого мы пришли из левого поддерева, и будет ответом.

8.5 Вставка

```

1  insert(root, x):
2      if root == null:
3          return new Node(x)
4      if x < root.key:
5          root.left = insert(root.left, x)
6      else if x > root.key:

```



```

7         root.right = insert(root.right, x)
8     return root

```

Корректность вставки

Если до вставки дерево было BST, то после вставки нового ключа оно остаётся BST.

Доказательство. Алгоритм спускается в левое поддерево только при $x < x_v$, а в правое — только при $x > x_v$. Поэтому для каждого предка, из которого путь ушёл влево, выполнено $x < x_v$, а для каждого предка, из которого путь ушёл вправо, выполнено $x > x_v$. Новый ключ помещается в лист, где все эти ограничения одновременно выполнены. Остальные поддеревья не изменяются. Следовательно, BST-инвариант сохраняется во всех вершинах. $\square$

Время вставки равно $O(h)$, так как изменяется только один путь от корня до новой вершины. В обычном BST это $O(n)$ в худшем случае и $O(\log n)$ при малой высоте.

8.6 Удаление

Удаление ключа x сначала находит соответствующую вершину v , а затем разбирает три случая.

1. У v нет детей. Тогда вершину можно просто удалить.
2. У v ровно один ребёнок. Тогда ребёнок подцепляется на место v .
3. У v два ребёнка. Тогда ключ v заменяется на следующий ключ $\text{next}(v)$, после чего удаляется вершина $\text{next}(v)$. У этой вершины нет левого ребёнка, поэтому её удаление сводится к одному из первых двух случаев.

```

1  erase(root, x):
2      if root == null:
3          return null
4      if x < root.key:
5          root.left = erase(root.left, x)
6      else if x > root.key:
7          root.right = erase(root.right, x)
8      else:
9          if root.left == null:
10             return root.right
11          if root.right == null:
12             return root.left
13          p = root.right
14          while p.left != null:
15              p = p.left
16          root.key = p.key
17          root.right = erase(root.right, p.key)
18      return root

```

Корректность удаления

После удаления ключа из BST оставшееся дерево снова является BST и содержит ровно прежние ключи без удалённого.

Доказательство. Первые два случая очевидны: если у вершины нет ребёнка или есть один ребёнок, то подцепляемое поддерево уже удовлетворяет ограничениям всех предков. В случае двух детей берётся следующий элемент $s = \text{next}(v)$. Он больше всех ключей левого поддерева v и не больше всех ключей правого поддерева, поскольку является минимумом правого поддерева. Поэтому замена ключа v на s сохраняет BST-инвариант. Затем исходная вершина с ключом s удаляется из правого поддерева; у неё нет левого ребёнка, значит удаление сводится к простому случаю. $\square$

Удаление работает за $O(h)$: поиск вершины занимает $O(h)$, поиск минимума в правом поддерева также идёт вниз по одному пути, и последующее удаление не выходит за тот же путь.

8.7 Lower bound и upper bound

В упорядоченном множестве часто нужны не только поиск точного ключа, но и ближайшие элементы. В BST они реализуются одним спуском.

Lower bound и upper bound

$\text{lower_bound}(x)$ — минимальный ключ y , такой что $y \geq x$. $\text{upper_bound}(x)$ — минимальный ключ y , такой что $y > x$.

```

1 lower_bound(v, x):
2     ans = null
3     while v != null:
4         if v.key >= x:
5             ans = v
6             v = v.left
7         else:
8             v = v.right
9     return ans

```

Время работы равно $O(h)$. Эти операции объясняют, почему BST удобно использовать как основу для упорядоченных множеств и словарей: можно искать элементы по порядку, переходить к следующему элементу и перечислять диапазон ключей.

Итераторы. Если хранить указатели на родителей, то вершина дерева сама может играть роль итератора: операции next и prev переходят к соседним ключам за $O(h)$. В реальных контейнерах часто поддерживают двусвязную прошивку по симметричному обходу, тогда переход к следующему и предыдущему элементу работает за $O(1)$, а вставка и удаление обновляют только локальные ссылки. Именно поэтому итераторы в упорядоченных множествах обычно остаются корректными для большинства не затронутых элементов после модификаций.

8.8 Высота дерева и балансировка

Все основные операции обычного BST имеют время $O(h)$, где h — высота дерева. Поэтому обычное BST само по себе не гарантирует логарифмическое время:

$$h = \Theta(n) \quad \Rightarrow \quad \text{find, insert, erase} = O(n).$$

Чтобы получить гарантию $O(\log n)$, используют сбалансированные деревья поиска и декартовы деревья. В них после модификаций поддерживается дополнительный инвариант, ограничивающий высоту. Например, в AVL-дереве для каждой вершины высоты двух детей отличаются не более чем на 1, откуда следует $h = O(\log n)$.

Дополнительные данные в вершинах. BST можно расширять: хранить в вершине размер поддерева, минимум/сумму по поддереву, отложенные операции. Тогда появляются операции порядковых статистик, запросы на отрезках ключей и структуры по неявному ключу. При корректной поддержке этих значений время операций остаётся пропорциональным высоте дерева.

9 Задача балансировки бинарных деревьев. Красно-чёрные деревья: определение, вычислительная сложность операций поиска, вставки и удаления.

9.1 Зачем нужна балансировка

Обычное BST сохраняет порядок ключей, но его высота зависит от порядка вставок. Если ключи вставлять в отсортированном порядке, дерево вырождается в цепочку:

$$h = \Theta(n), \quad \text{find, insert, erase} = \Theta(n).$$

Сбалансированное дерево поиска

Сбалансированное дерево поиска — это BST, в котором поддерживается дополнительный инвариант, гарантирующий высоту

$$h = O(\log n).$$

Тогда все операции, выполняющие один спуск по дереву, работают за $O(\log n)$.

Балансировку поддерживают локальными изменениями ссылок. Главная локальная операция — вращение. Оно меняет форму дерева, но сохраняет симметричный порядок ключей.

```
1 rotateLeft(x):  
2     y = x.right  
3     x.right = y.left  
4     y.left = x  
5     return y
```

Если до левого вращения порядок поддеревьев был

$$A < x < B < y < C,$$

то после вращения он остаётся тем же:

$$A < x < B < y < C.$$

Поэтому вращения сохраняют BST-инвариант.

9.2 Определение красно-чёрного дерева

Красно-чёрное дерево

Красно-чёрное дерево — это BST, в котором каждая вершина покрашена в красный или чёрный цвет. Все отсутствующие дети считаются фиктивными чёрными листьями NIL. Должны выполняться свойства:

- корень чёрный;
- все фиктивные листья NIL чёрные;
- у красной вершины оба ребёнка чёрные;
- на любом пути от данной вершины до фиктивного листа одинаковое число чёрных вершин.

Чёрной высотой $bh(v)$ будем называть число чёрных вершин на любом пути от v до фиктивного листа, не считая саму вершину v , но считая конечный лист NIL. Красные вершины можно понимать как часть кодирования более толстых вершин 2-3-4-дерева: если слить красную вершину с её чёрным отцом, получаются вершины с 1, 2 или 3 ключами. Поэтому красно-чёрное дерево является бинарной реализацией сбалансированного 2-3-4-дерева.

9.3 Оценка высоты

Высота красно-чёрного дерева

Красно-чёрное дерево с n внутренними вершинами имеет высоту

$$h \leq 2 \log_2(n + 1).$$

Доказательство. Сначала докажем, что поддереву с корнем v содержит хотя бы

$$2^{bh(v)} - 1$$

внутренних вершин. Доказательство идёт индукцией по высоте поддерева. Для фиктивного листа утверждение очевидно. У внутренней вершины каждый ребёнок имеет чёрную высоту не меньше $bh(v) - 1$, поэтому по индукции два поддерева вместе содержат хотя бы

$$2(2^{bh(v)-1} - 1)$$

вершин, а вместе с самой вершиной получаем

$$1 + 2(2^{bh(v)-1} - 1) = 2^{bh(v)} - 1.$$

Для корня r отсюда следует

$$n \geq 2^{bh(r)} - 1, \quad bh(r) \leq \log_2(n + 1).$$

На любом пути не может быть двух красных вершин подряд, значит число красных вершин на пути не превосходит числа чёрных. Поэтому обычная высота не больше удвоенной чёрной:

$$h \leq 2bh(r) \leq 2 \log_2(n + 1).$$

□

9.4 Поиск

Поиск в красно-чёрном дереве ничем не отличается от поиска в обычном BST: цвет вершины не используется.

```
1 find(v, x):
2     while v != NIL and v.key != x:
3         if x < v.key:
4             v = v.left
5         else:
6             v = v.right
7     return v
```

Так как высота красно-чёрного дерева равна $O(\log n)$, поиск работает за

$$O(\log n).$$

9.5 Вставка

Вставка начинается как в обычном BST: новый ключ вставляется листом. Новую вершину красят в красный цвет. Это не меняет чёрные высоты путей, но может нарушить правило: у красной вершины не должно быть красного отца.

Сложность вставки

Вставка в красно-чёрное дерево работает за $O(\log n)$. При восстановлении инвариантов выполняется $O(\log n)$ перекрасок и не более двух вращений.

Доказательство. Спуск к месту вставки занимает $O(h) = O(\log n)$. После вставки возможен конфликт только между новой красной вершиной и её красным отцом. Рассмотрим отца p , деда g и дядю u .

Если u красный, то p и u перекрашиваются в чёрный, а g — в красный. Чёрная высота поддерева с корнем g сохраняется, но конфликт может подняться к отцу g . Значит таких шагов может быть не больше высоты дерева.

Если u чёрный, то конфликт устраняется одним или двумя вращениями около g и перекраской новых локальных корней. После этого красный родитель у красной вершины исчезает, а чёрные высоты путей сохраняются, поэтому подъём дальше не нужен.

В конце корень перекрашивается в чёрный. Все действия происходят на пути к корню, поэтому время равно $O(\log n)$, а вращений в последнем случае нужно не больше двух. $\square$

9.6 Удаление

Удаление сначала выполняется как в обычном BST. Если у удаляемой вершины два ребёнка, её ключ меняют с ключом следующей вершины, после чего реально удаляется вершина с не более чем одним нефиктивным ребёнком.

Если удалённая вершина была красной, инварианты не нарушаются. Сложность появляется при удалении чёрной вершины: на одном из путей становится на одну чёрную вершину меньше. Это состояние часто называют «двойной чёрнотой». Его исправляют, рассматривая брата текущей вершины, цвета брата и его детей; в зависимости от случая выполняются перекраски и одно или несколько вращений. Лишняя чёрнота либо исчезает локально, либо поднимается к родителю.

Сложность удаления

Удаление из красно-чёрного дерева работает за $O(\log n)$. Восстановление инвариантов делает $O(\log n)$ перекрасок и $O(1)$ вращений.

Доказательство. Поиск удаляемой вершины и, при необходимости, её следующего элемента занимает $O(\log n)$. После физического удаления нарушение может подниматься только вверх по пути к корню: на каждом шаге рассматриваются родитель, брат и дети брата, то есть константное число вершин.

Если нарушение не устраняется сразу вращением и перекраской, оно переносится на уровень выше. Таких переносов не больше высоты дерева, то есть $O(\log n)$. В стандартном разборе случаев удаление требует не более константного числа вращений, а перекраски могут происходить на каждом уровне. Следовательно, полное время удаления равно $O(\log n)$. $\square$

9.7 Итоговые оценки

Операция	Время в красно-чёрном дереве
find	$O(\log n)$
lower_bound, upper_bound	$O(\log n)$
insert	$O(\log n)$
erase	$O(\log n)$
next, prev без прошивки	$O(\log n)$

Красно-чёрные деревья используются как практичная реализация упорядоченных множеств и словарей. Например, контейнеры `set` и `map` в C++ обычно реализуются именно через красно-чёрное дерево: они поддерживают порядок ключей, итераторы и логарифмические операции.

10 Задача хеширования. Хеш-функции. Таблица с прямой адресацией.

10.1 Задача словаря

Хеширование применяется для реализации динамического словаря: изначально есть пустое множество или отображение, и требуется поддерживать операции

$$\text{insert}(x), \quad \text{erase}(x), \quad \text{find}(x).$$

Для ассоциативного массива вместо одного ключа x хранится пара $(key, value)$, а поиск выполняется по ключу.

Словарь

Словарь — структура данных, поддерживающая хранение элементов по ключам. В неупорядоченном словаре не требуется уметь находить следующий ключ или перечислять ключи в порядке возрастания. Поэтому хеш-таблица может быть быстрее BST: она целится в $O(1)$ на основные операции, но не поддерживает порядок.

Наивная реализация через вектор имеет быстрый insert в конец, но медленный поиск:

$$\text{insert} = O(1), \quad \text{find} = O(n), \quad \text{erase} = O(n).$$

Если порядок элементов не важен, удаление после найденной позиции можно сделать обменом с последним элементом и pop_back, но поиск всё равно остаётся линейным.

10.2 Прямая адресация

Таблица с прямой адресацией

Пусть все возможные ключи лежат в небольшом множестве

$$U = \{0, 1, \dots, m - 1\}.$$

Таблица с прямой адресацией хранит массив $T[0..m - 1]$, где ячейка $T[x]$ отвечает непосредственно за ключ x .

Для множества достаточно булевого массива:

```
1 insert(x):  
2     used[x] = true  
3  
4 erase(x):  
5     used[x] = false  
6  
7 find(x):  
8     return used[x]
```

Все операции работают за $O(1)$ в худшем случае. Для отображения вместо `used[x]` можно хранить значение и отдельный признак занятости.

Свойства прямой адресации

Таблица с прямой адресацией даёт $O(1)$ на insert, erase и find в худшем случае, но требует $\Theta(|U|)$ памяти.

Доказательство. Адрес нужной ячейки вычисляется непосредственно по ключу x , поэтому каждая операция выполняет постоянное число обращений к массиву. Память выделяется под все ключи из универсума, включая те, которые никогда не появятся, поэтому требуется $\Theta(|U|)$ ячеек. $\square$

Главная проблема прямой адресации — большой универсум. Например, если ключи являются 64-битными числами, массив размера 2^{64} невозможен, даже если реально хранится всего несколько тысяч элементов.

10.3 Хеш-функция

Хеш-функция

Хеш-функция — отображение

$$h : U \rightarrow \{0, 1, \dots, m - 1\},$$

которое переводит большой универсум ключей в индексы таблицы размера m .

Идея хеш-таблицы состоит в том, чтобы хранить ключ x не в ячейке x , а около ячейки $h(x)$. Поэтому память зависит от числа реально хранимых элементов, а не от размера универсума.

Коллизия

Коллизия — ситуация, когда для разных ключей $x \neq y$ выполнено

$$h(x) = h(y).$$

Коллизии неизбежны, если $|U| > m$, поэтому хеш-таблица должна иметь способ их разрешения. Два основных подхода: хранить в каждой ячейке список элементов с таким хешем или искать другую свободную ячейку в самой таблице.

Примеры хеш-функций. Для целых чисел часто берут

$$h(x) = x \bmod m.$$

Если m выбрано неудачно, например является степенью двойки, то функция может зависеть только от младших битов ключа. Поэтому на практике часто берут простое m или используют дополнительное случайное умножение:

$$h(x) = ((a \cdot x + b) \bmod p) \bmod m,$$

где p — простое число, большее всех возможных ключей, а a, b выбираются случайно.

Универсальное семейство хеш-функций

Семейство хеш-функций $\mathcal{H}$ называется универсальным, если для любых различных ключей $x \neq y$

$$\Pr_{h \in \mathcal{H}}[h(x) = h(y)] \leq \frac{1}{m}.$$

Случайный выбор функции из такого семейства защищает от фиксированного набора плохих ключей: для любой пары вероятность коллизии не больше, чем при равномерном случайном распределении по m ячейкам.

10.4 Коэффициент заполнения и rehash

Коэффициент заполнения

Если в таблице размера m хранится n элементов, то величина

$$\alpha = \frac{n}{m}$$

называется коэффициентом заполнения.

Чем больше α , тем чаще возникают коллизии. Обычно поддерживают ограничение $\alpha \leq C$, где C зависит от способа разрешения коллизий. При переполнении делают rehash: создают таблицу большего размера и заново вставляют все элементы.

Амортизированное время rehash

Если размер таблицы увеличивать в константное число раз, то стоимость перевыделений даёт $O(1)$ амортизированной добавки к операции вставки.

Доказательство. При увеличении таблицы с размера m до $2m$ требуется $O(n)$ времени на перенос элементов. До следующего увеличения произойдёт $\Omega(m)$ вставок, поэтому стоимость переноса можно распределить между ними по $O(1)$ на операцию. $\square$

11 Разрешение коллизий при хешировании. Открытая адресация.

11.1 Идея открытой адресации

Открытая адресация

Хеш-таблица с открытой адресацией хранит все элементы непосредственно в одном массиве. Если ячейка $h(x)$ занята, алгоритм просматривает другие ячейки по некоторой последовательности проб, пока не найдёт ключ x или подходящее свободное место.

Самый простой вариант — линейное пробирование:

$$h_i(x) = (h(x) + i) \bmod m, \quad i = 0, 1, 2, \dots$$

То есть после начальной ячейки алгоритм двигается вправо по циклическому массиву.

```
1 getIndex(x):
2     i = h(x)
3     while table[i] is occupied and table[i].key != x:
4         i = (i + 1) mod m
5     return i
```

11.2 Состояния ячеек

Для корректной работы удаления одной отметки «пусто» недостаточно. Обычно используют три состояния ячеек:

- EMPTY: в этой ячейке никогда не было элемента после последнего rehash;
- USED: в ячейке сейчас лежит элемент;
- DELETED: элемент был удалён, но поиск через эту ячейку должен продолжаться.

Если при удалении просто сделать ячейку пустой, можно сломать поиск ключей, которые были поставлены правее из-за коллизий. Поэтому удалённая ячейка становится «могилкой» DELETED. При вставке её можно переиспользовать, а при поиске через неё нужно проходить дальше.

```

1 find(x):
2     i = h(x)
3     while table[i] != EMPTY:
4         if table[i] == USED and table[i].key == x:
5             return true
6         i = (i + 1) mod m
7     return false

```

11.3 Сложность линейного пробирования

Оценка времени поиска

Пусть таблица с открытой адресацией имеет размер m , а доля ячеек, через которые поиск не может остановиться, равна $\alpha < 1$. Сюда входят как USED, так и DELETED-ячейки. Если хеш-функция распределяет ключи достаточно равномерно, то ожидаемое число просмотренных ячеек при поиске свободной позиции оценивается как

$$O\left(\frac{1}{1-\alpha}\right).$$

Доказательство. В худшем для оценки случае искомого ключа нет, и поиск останавливается на первой свободной ячейке. При равномерном распределении вероятность того, что очередная просмотренная ячейка не остановит поиск, равна примерно α . Поэтому вероятность сделать хотя бы k -й шаг не превосходит α^k . Математическое ожидание числа просмотренных ячеек:

$$1 + \alpha + \alpha^2 + \alpha^3 + \dots = \frac{1}{1-\alpha}.$$

□

Из оценки видно, что при α , близком к 1, операции становятся медленными. Например, при $\alpha = 0.9$ ожидаемая длина проб уже порядка 10. Поэтому для открытой адресации обычно поддерживают таблицу заметно незаполненной, например делают rehash при $\alpha > \frac{2}{3}$ или при $\alpha > \frac{3}{4}$.

11.4 Виды пробирования

Линейное пробирование. Последовательность проб:

$$h_i(x) = (h(x) + i) \bmod m.$$

Плюсы: простая реализация и хорошая локальность памяти. Минус: первичная кластеризация. Если образовался длинный непрерывный блок занятых ячеек, новые элементы рядом с ним будут удлинять этот же блок.

Квадратичное пробирование. Последовательность проб:

$$h_i(x) = (h(x) + c_1 i + c_2 i^2) \bmod m.$$

Оно уменьшает первичную кластеризацию, но требует аккуратного выбора параметров и размера таблицы, чтобы последовательность могла найти свободную ячейку.

Двойное хеширование. Используются две хеш-функции:

$$h_i(x) = (h_1(x) + i \cdot h_2(x)) \bmod m.$$

Если $h_2(x)$ взаимно просто с m , последовательность проб может пройти всю таблицу. Это обычно даёт более равномерные пробы, но чуть дороже вычислительно.

11.5 Удаления и накопление могил

Метки DELETED сохраняют корректность поиска, но ухудшают время работы: для поиска они ведут себя как занятые ячейки. Поэтому при большом числе удалений нужно периодически пересобирать таблицу, заново вставляя только живые элементы. Такой rehash одновременно увеличивает таблицу при переполнении и очищает могилки.

Свойства открытой адресации

Хеш-таблица с открытой адресацией использует один массив и имеет хорошую локальность памяти. При хорошем хешировании и ограниченном α операции работают за ожидаемое $O(1)$, но в худшем случае могут занять $O(n)$.

12 Разрешение коллизий в хеш-таблицах на основе списков.

12.1 Метод цепочек

Хеш-таблица на цепочках

В хеш-таблице на цепочках каждая ячейка массива хранит контейнер, обычно список, всех элементов с одинаковым значением хеш-функции:

$$bucket(x) = h(x).$$

```
1 insert(x):
2     b = h(x)
3     if x not in table[b]:
4         table[b].push_front(x)
5
6 find(x):
7     b = h(x)
8     scan table[b]
9
10 erase(x):
11     b = h(x)
12     erase x from table[b]
```

Вместо связного списка можно использовать вектор, маленький массив, дерево поиска или другую структуру. На практике для коротких цепочек вектор часто быстрее списка из-за лучшей локальности памяти.

12.2 Оценка времени

Пусть в таблице m корзин и n элементов. Обозначим длину корзины i через L_i . Тогда

$$\sum_{i=0}^{m-1} L_i = n, \quad \mathbb{E}[L_{h(x)}] \approx \frac{n}{m} = \alpha.$$

Среднее время операций в таблице на цепочках

Если хеш-функция распределяет ключи равномерно, то ожидаемая длина цепочки равна $\alpha = n/m$. Поэтому операции `find`, `insert` и `erase` работают за $O(1 + \alpha)$ в среднем.

Доказательство. Операция сначала вычисляет индекс корзины за $O(1)$, а затем просматривает только одну цепочку. При равномерном распределении ожидаемое число элементов в выбранной цепочке равно $n/m = \alpha$. Поэтому итоговое ожидаемое время равно $O(1 + \alpha)$. $\square$

Если поддерживать $\alpha = O(1)$, например увеличивать число корзин при $n > m$, то получается ожидаемое $O(1)$ на операцию. В худшем случае все элементы могут попасть в одну цепочку, и тогда поиск станет $O(n)$.

12.3 Перевыделение таблицы

При росте числа элементов поддерживают ограничение на коэффициент заполнения:

$$\alpha = \frac{n}{m} \leq C.$$

Когда оно нарушается, создают новый массив корзин, обычно в два раза больше, и передо-
бавляют все элементы. Важно именно заново вычислить $h(x)$ по новому размеру таблицы: старые номера корзин уже не подходят.

```
1 rehash():
2     old = table
3     table = new buckets[2 * m]
4     m = 2 * m
5     for each bucket in old:
6         for each x in bucket:
7             table[h(x)].push_front(x)
```

Как и для расширяющегося массива, редкие линейные пересборки дают $O(1)$ амортизированной добавки на вставку.

12.4 Сравнение с открытой адресацией

Свойство	Цепочки	Открытая адресация
Где лежат элементы	В отдельных контейнерах по корзинам	Непосредственно в массиве таблицы
Удаление	Простое: удалить из цепочки	Нужны состояния EMPTY, USED, DELETED
Память	Дополнительные указатели или служебная память контейнеров	Обычно компактнее, но нужен запас пустых ячеек
Локальность памяти	Хуже для связанных списков	Лучше: просмотр идёт по массиву
Поведение при большом α	Цепочки становятся длиннее	Пробы резко удлиняются, при $\alpha = 1$ вставка невозможна

12.5 Практические замечания

В C++ неупорядоченные хеш-таблицы представлены двумя основными контейнерами:

- `unordered_set` — множество ключей;
- `unordered_map` — ассоциативный массив ключ-значение.

Если примерное число элементов известно заранее, полезно вызвать `reserve`: это уменьшает число `rehash`-операций.

Хеш-таблица даёт быстрые операции только при хорошей хеш-функции. Если тесты могут быть подобраны против программы, полезно добавлять случайность в хеширование, например использовать случайный множитель или случайную соль. Это защищает от антихеш-тестов, когда много разных ключей специально попадают в одни и те же корзины.

13 Графы. Ориентированные и неориентированные графы, деревья. Пути, циклы. Задача связности. Представление графов. Лемма о рукопожатиях.

13.1 Основные определения

Граф

Графом называется пара

$$G = (V, E),$$

где V — множество вершин, а E — множество рёбер. В неориентированном графе ребро задаётся неупорядоченной парой $\{u, v\}$, в ориентированном графе — упорядоченной парой (u, v) , которую также называют дугой.

Если рёбрам сопоставлены числа, граф называется взвешенным. Если допускаются петли (v, v) и несколько одинаковых рёбер между одной парой вершин, говорят о мультиграфе. Простой граф — граф без петель и кратных рёбер.

Степени вершин

В неориентированном графе степень вершины $\deg v$ — число рёбер, инцидентных v . В ориентированном графе различают входящую и исходящую степени:

$$\deg v = \deg^{in} v + \deg^{out} v.$$

Путь и цикл

Путь — последовательность вершин

$$v_0, v_1, \dots, v_k,$$

такая что между v_i и v_{i+1} есть ребро. В ориентированном графе требуется ребро (v_i, v_{i+1}) . Путь называется простым, если все его вершины различны. Цикл — путь положительной длины, у которого начало и конец совпадают.

Дерево

Дерево — связный неориентированный граф без циклов.

Для графа с n вершинами следующие свойства эквивалентны:

граф является деревом

$\iff$ он связен и имеет $n - 1$ ребро

$\iff$ между любыми двумя вершинами есть ровно один простой путь.

Лемма о рукопожатиях

В любом неориентированном графе

$$\sum_{v \in V} \deg v = 2|E|.$$

В ориентированном графе

$$\sum_{v \in V} \deg^{in} v = \sum_{v \in V} \deg^{out} v = |E|.$$

Доказательство. В неориентированном графе каждое ребро имеет два конца и поэтому даёт вклад 1 в степень каждой из двух инцидентных вершин, суммарно вклад 2. В ориентированном графе каждая дуга имеет ровно одно начало и один конец, поэтому она один раз учитывается в сумме исходящих степеней и один раз в сумме входящих степеней. $\square$

13.2 Связность

В неориентированном графе вершины u и v называются связанными, если между ними существует путь. Это отношение эквивалентности: оно рефлексивно, симметрично и транзитивно. Классы эквивалентности называются компонентами связности.

В ориентированном графе различают достижимость и сильную связность. Вершина v достижима из u , если существует ориентированный путь $u \rightsquigarrow v$. Вершины u и v сильно связны, если $u \rightsquigarrow v$ и $v \rightsquigarrow u$.

13.3 Представление графов

Пусть $n = |V|$, $m = |E|$.

Представление	Память	Проверка ребра	Перебор соседей вершины v
Список рёбер	$O(m)$	$O(m)$	$O(m)$
Матрица смежности	$O(n^2)$	$O(1)$	$O(n)$
Списки смежности	$O(n + m)$	$O(\deg v)$	$O(\deg v)$

Матрица смежности удобна для плотных графов и быстрых запросов вида “есть ли ребро (u, v) ”. Списки смежности обычно лучше для разреженных графов и обходов, потому что алгоритмы просматривают только реально существующие рёбра.

14 Графы. Обход графа в ширину. Корректность и сложность алгоритма. Структура дерева поиска. Примеры использования алгоритма.

14.1 Идея BFS

Обход в ширину, или BFS, решает задачу поиска расстояний от стартовой вершины s в невзвешенном графе. Расстояние здесь равно минимальному числу рёбер в пути.

```
1 BFS(s):
2     for v in V:
3         dist[v] = INF
4         parent[v] = -1
5     dist[s] = 0
6     q.push(s)
7     while q is not empty:
8         v = q.pop()
9         for x in g[v]:
10             if dist[x] == INF:
11                 dist[x] = dist[v] + 1
12                 parent[x] = v
13                 q.push(x)
```

Очередь хранит вершины в порядке неубывания расстояния. Массив `parent` задаёт дерево поиска в ширину: если вершина $v \neq s$ достижима, то `parent[v]` — предыдущая вершина на найденном кратчайшем пути из s в v .

Корректность BFS

После выполнения BFS для каждой достижимой из s вершины v значение `dist[v]` равно длине кратчайшего пути из s в v .

Доказательство. Докажем по индукции по расстоянию. Для s расстояние равно 0, и алгоритм сразу записывает `dist[s] = 0`.

Пусть для всех вершин на расстоянии не больше d значения уже найдены корректно и такие вершины попали в очередь не позже вершин большего расстояния. Возьмём вершину x на расстоянии $d + 1$. На кратчайшем пути к ней есть предыдущая вершина v на расстоянии d . Когда алгоритм обработает v , он рассмотрит ребро (v, x) и, если x ещё не посещена, присвоит $\text{dist}[x] = d + 1$.

Меньшее значение алгоритм присвоить не может: если вершина получает расстояние $\text{dist}[v] + 1$, то существует путь такой длины через v . Очередь обрабатывает вершины по слоям, поэтому первое присвоенное значение является минимальным. $\square$

14.2 Сложность

При хранении графа списками смежности каждая вершина попадает в очередь не более одного раза, а каждое ребро просматривается не более константного числа раз. Поэтому

$$T(n, m) = O(n + m).$$

Память также равна $O(n + m)$ для графа и $O(n)$ для очереди, расстояний и родителей.

14.3 Применения BFS

- поиск кратчайших путей в невзвешенном графе;
- восстановление кратчайшего пути по массиву `parent`;
- проверка связности неориентированного графа;
- нахождение компонент связности запуском BFS из каждой ещё не посещённой вершины;
- проверка двудольности графа раскраской вершин по чётности расстояния.

Проверка двудольности

Неориентированный граф двудолен тогда и только тогда, когда при BFS-раскраске по чётности расстояния нет ребра, соединяющего вершины одного цвета.

Доказательство. Если есть ребро между вершинами одного цвета, то вместе с путями от корня BFS до этих вершин оно образует нечётный цикл, а граф с нечётным циклом не двудолен. Если таких рёбер нет, то все рёбра идут между разными цветами, значит полученная раскраска является двудольной. $\square$

15 Графы. Обход графа в глубину. Корректность и сложность алгоритма. Структура дерева поиска. Примеры использования алгоритма.

15.1 Идея DFS

Обход в глубину, или DFS, рекурсивно уходит по ещё не посещённым рёбрам, пока это возможно.


```

1 DFS(v):
2     used[v] = true
3     tin[v] = timer++
4     for x in g[v]:
5         if not used[x]:
6             parent[x] = v
7             DFS(x)
8     tout[v] = timer++

```

Если нужно обойти весь граф, DFS запускают от каждой ещё не посещённой вершины. Рёбра, по которым DFS впервые входит в новые вершины, образуют лес DFS. Для одной компоненты связности это дерево DFS.

Корректность DFS для достижимости

После вызова $\text{DFS}(s)$ отмечены ровно те вершины, которые достижимы из s .

Доказательство. DFS переходит только по рёбрам графа, поэтому любая отмеченная вершина достижима из s по пути рекурсивных вызовов. Обратно, пусть вершина u достижима из s , и рассмотрим путь $s = v_0, v_1, \dots, v_k = u$. Если DFS отметил v_i , но не отметил v_{i+1} , то при обработке v_i алгоритм рассмотрел ребро к v_{i+1} и должен был рекурсивно перейти в неё. Противоречие. Значит все вершины на пути будут отмечены. $\square$

15.2 Сложность

При списках смежности каждая вершина обрабатывается один раз, а каждое ребро просматривается не более константного числа раз:

$$T(n, m) = O(n + m).$$

Дополнительная память равна $O(n)$ для массивов и стека рекурсии.

15.3 Времена входа и выхода

Времена tin и tout позволяют проверять отношение предок-потомок в дереве DFS:

$$u \text{ является предком } v \iff \text{tin}[u] \leq \text{tin}[v] \text{ и } \text{tout}[v] \leq \text{tout}[u].$$

В ориентированном графе рёбра относительно дерева DFS делят на древесные, обратные, прямые и перекрёстные. Обратное ребро ведёт из вершины в её предка и является главным признаком цикла в ориентированном графе.

В неориентированном DFS нет перекрёстных рёбер

Относительно дерева DFS неориентированного графа каждое недревесное ребро соединяет вершину с её предком.

Доказательство. Пусть есть ребро $\{a, b\}$, соединяющее разные поддеревья DFS, и $\text{tin}[a] < \text{tin}[b]$. Тогда при обработке a вершина b ещё не была завершена и должна была быть достигнута через это ребро из поддерева a . Значит b не могла оказаться в другом поддереве. Противоречие. $\square$

15.4 Применения DFS

- поиск компонент связности;
- восстановление пути по массиву родителей;
- проверка наличия цикла;
- топологическая сортировка ориентированного ациклического графа;
- поиск мостов, точек сочленения и компонент сильной связности.

16 Графы. Топологическая сортировка. Корректность и сложность алгоритма. Поиск циклов в графе.

16.1 Определение

DAГ и топологический порядок

DAГ — ориентированный ациклический граф. Топологическая сортировка ориентированного графа — такой порядок вершин, что для каждого ребра (u, v) вершина u стоит раньше вершины v .

Критерий существования топологической сортировки

Топологическая сортировка существует тогда и только тогда, когда ориентированный граф ациклический.

Доказательство. Если в графе есть цикл $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k \rightarrow v_1$, то в топологическом порядке должны выполняться неравенства

$$v_1 < v_2 < \dots < v_k < v_1,$$

что невозможно.

Обратно, пусть граф ациклический. В любом конечном DAГ есть вершина с нулевой входящей степенью: иначе, двигаясь каждый раз по входящему ребру назад, мы из-за конечности вернулись бы в уже посещённую вершину и получили цикл. Удалим такую вершину и по индукции построим топологический порядок для оставшегося графа. Поставив удалённую вершину первой, получим корректный порядок для всего графа. $\square$

16.2 Алгоритм Кана

```
1 TopSortKahn():
2   q = all vertices with indeg[v] == 0
3   order = []
4   while q is not empty:
5       v = q.pop()
6       order.push_back(v)
7       for x in g[v]:
8           indeg[x]--
9           if indeg[x] == 0:
10              q.push(x)
11   if |order| < n:
```

Алгоритм работает за $O(n + m)$: каждая вершина один раз попадает в очередь, каждое ребро один раз уменьшает входящую степень своего конца.

16.3 Топологическая сортировка через DFS

Если запустить DFS и выписывать вершину в момент выхода, то обратный порядок выхода даёт топологическую сортировку DAG.

```

1 DFS(v):
2     color[v] = 1
3     for x in g[v]:
4         if color[x] == 0:
5             DFS(x)
6     color[v] = 2
7     order.push_back(v)
8
9 reverse(order)
```

Корректность DFS-топсорта

В ациклическом ориентированном графе для каждого ребра (u, v) выполнено

$$\text{tout}[u] > \text{tout}[v].$$

Поэтому вершины в порядке убывания времени выхода образуют топологическую сортировку.

Доказательство. Рассмотрим ребро (u, v) . Если DFS впервые пришёл в u раньше, чем в v , то он либо перейдёт из u в v , либо v уже будет обработана из другой ветки. В обоих случаях v завершится раньше u . Если DFS впервые пришёл в v раньше, то u не может быть достижима из v , иначе вместе с ребром $u \rightarrow v$ возник бы цикл. Значит v снова завершится раньше u . Следовательно, $\text{tout}[u] > \text{tout}[v]$. $\square$

16.4 Поиск циклов

Для ориентированного графа удобно использовать цвета:

0 — не посещена, 1 — вершина в стеке DFS, 2 — обработана.

Если DFS видит ребро в серую вершину, найден ориентированный цикл. Если DFS завершился без таких рёбер, граф ациклический.

```

1 DFS(v):
2     color[v] = 1
3     for x in g[v]:
4         if color[x] == 1:
5             cycle found
6         if color[x] == 0:
7             DFS(x)
8     color[v] = 2
```

В неориентированном графе цикл находится похожим образом, но нужно игнорировать ребро, по которому DFS пришёл из родителя. Если DFS видит посещённую вершину, отличную от родителя, то найден цикл.

17 Графы. Поиск сильно связанных компонент. Корректность и сложность алгоритма.

17.1 Компоненты сильной связности

Сильная связность

В ориентированном графе вершины u и v сильно связны, если существует путь $u \rightsquigarrow v$ и существует путь $v \rightsquigarrow u$. Компонента сильной связности — класс эквивалентности этого отношения.

Если каждую компоненту сильной связности сжать в одну вершину, а рёбра между компонентами сохранить, получится граф конденсации.

Конденсация ациклична

Граф конденсации любого ориентированного графа является DAG.

Доказательство. Если бы в конденсации был цикл компонент

$$C_1 \rightarrow C_2 \rightarrow \dots \rightarrow C_k \rightarrow C_1,$$

то из любой компоненты этого цикла была бы достижима любая другая. Тогда все вершины компонент $C_1, \dots, C_k$ лежали бы в одной компоненте сильной связности, что противоречит максимальной компоненте. $\square$

17.2 Алгоритм Косарайю

Алгоритм Косарайю основан на двух обходах DFS и транспонировании графа. Обозначим через $\text{tout}[v]$ время выхода из вершины v в первом DFS.

Лемма о временах выхода

Пусть C и D — две разные компоненты сильной связности, и в конденсации есть ребро $C \rightarrow D$. Тогда

$$\max_{v \in C} \text{tout}[v] > \max_{v \in D} \text{tout}[v].$$

Доказательство. Рассмотрим, в какую из компонент DFS впервые попал раньше. Если первой была достигнута компонента C , то из неё достижима вся компонента D , поэтому DFS обработает D до выхода из вершины компоненты C . Значит максимальное время выхода в C больше максимального времени выхода в D . Если первой была достигнута компонента D , то из D нельзя попасть в C , иначе C и D лежали бы в одной КСС. Следовательно, DFS полностью выйдет из D до того, как начнёт обход C , и неравенство снова выполнено. $\square$

Из леммы следует, что если упорядочить вершины по убыванию времени выхода, то первой среди ещё не обработанных будет вершина из компоненты-источника конденсации ис-

ходного графа. После транспонирования графа эта компонента становится компонентой-стоком, поэтому DFS из неё не выйдет в другие ещё не выделенные компоненты.

```
1 Kosaraju():
2     used = false
3     order = []
4     for v in V:
5         if not used[v]:
6             DFS1(v) // in original graph, push v on exit
7
8     color = 0
9     reverse(order)
10    for v in order:
11        if color[v] == 0:
12            cc++
13            DFS2(v, cc) // in reversed graph
```

Шаги алгоритма:

1. Построить транспонированный граф G^T , то есть заменить каждое ребро $u \rightarrow v$ на ребро $v \rightarrow u$.
2. Запустить DFS в исходном графе и добавлять вершину в список order в момент выхода из неё.
3. Развернуть список order. Теперь вершины идут по убыванию tout.
4. В этом порядке запускать DFS в G^T из ещё не покрашенных вершин. Все вершины, достигнутые одним запуском, образуют одну компоненту сильной связности.

Корректность алгоритма Косарайю

Каждый запуск второго DFS в алгоритме Косарайю выделяет ровно одну компоненту сильной связности.

Доказательство. По лемме о временах выхода первая ещё не покрашенная вершина в порядке второго прохода лежит в компоненте-источнике конденсации исходного графа: в неё не входит ребро из другой непокрашенной компоненты. В транспонированном графе такая компонента становится стоком, поэтому DFS из неё не может перейти в другую непокрашенную компоненту.

С другой стороны, транспонирование не меняет разбиение на КСС: если $u \rightsquigarrow v$ и $v \rightsquigarrow u$ в исходном графе, то в обратном графе также есть пути в обе стороны, только направления всех путей меняются. Поэтому DFS в G^T , запущенный из вершины компоненты, обойдёт все вершины этой компоненты и не выйдет за её пределы. После её покраски то же рассуждение применяется к оставшимся компонентам. Следовательно, алгоритм выделяет все компоненты сильной связности корректно. $\square$

17.3 Сложность и применения

Построение обратного графа занимает $O(n + m)$. Каждый из двух DFS также работает за $O(n + m)$, поэтому суммарная сложность алгоритма Косарайю:

$$O(n + m).$$

Память равна $O(n + m)$.

Компоненты сильной связности используются, когда нужно заменить произвольный ориентированный граф на DAG: после сжатия компонент можно применять динамику по топологическому порядку, искать достижимость между компонентами и анализировать структуру зависимостей.

18 Графы. Задача поиска кратчайшего расстояния в графе. Алгоритм Дейкстры. Корректность и сложность алгоритма.

18.1 Постановка задачи

Кратчайший путь

Пусть дан взвешенный граф $G = (V, E)$, каждому ребру e сопоставлен вес $w(e)$. Вес пути — сумма весов его рёбер. Расстояние $\text{dist}(s, v)$ — минимальный вес пути из s в v , если такой путь существует; иначе расстояние равно $+\infty$.

Основная задача в этом билете — SSSP (*single-source shortest paths*): по заданной стартовой вершине s найти расстояния от s до всех остальных вершин. Для невзвешенного графа эту задачу решает BFS за $O(n + m)$, потому что все рёбра имеют одинаковую стоимость. Для графа с неотрицательными весами используется алгоритм Дейкстры.

Релаксация ребра

Для ребра $e = (v, x)$ релаксация — попытка улучшить текущее расстояние до x через вершину v :

если $d[x] > d[v] + w(v, x)$, то $d[x] := d[v] + w(v, x)$.

Если нужно восстановить путь, вместе с обновлением сохраняют $\text{parent}[x] = v$.

18.2 Алгоритм Дейкстры

Алгоритм поддерживает множество уже обработанных вершин A . На каждом шаге выбирается необработанная вершина с минимальной текущей оценкой расстояния, после чего релаксируются все исходящие из неё рёбра.

```
1 Dijkstra(s):
2   for v in V:
3       d[v] = INF
4       parent[v] = -1
5   d[s] = 0
6   priority_queue q
7   q.push((0, s))
8
9   while q is not empty:
10      (dist_v, v) = q.extractMin()
11      if dist_v != d[v]:
12          continue
13      for edge (v, x, w) in g[v]:
```

```

14         if d[x] > d[v] + w:
15             d[x] = d[v] + w
16             parent[x] = v
17             q.push((d[x], x))

```

В варианте с обычной бинарной кучей часто не делают настоящую операцию `decreaseKey`, а просто кладут в очередь новую пару $(d[x], x)$. Старые пары отбрасываются проверкой `dist_v != d[v]`.

Корректность алгоритма Дейкстры

Если все веса рёбер неотрицательны, то в момент извлечения вершины v с минимальным значением $d[v]$ из очереди значение $d[v]$ равно истинному кратчайшему расстоянию $\text{dist}(s, v)$.

Доказательство. Докажем утверждение индукцией по числу обработанных вершин. Пусть A — множество вершин, уже извлечённых из очереди и окончательно обработанных. Инвариант: для каждой вершины $u \in A$ значение $d[u]$ равно истинному расстоянию от s .

Рассмотрим очередную вершину $v \notin A$ с минимальным $d[v]$. Предположим, что существует путь из s в v меньшего веса. Рассмотрим на этом пути первое ребро (a, b) , выходящее из множества A : вершина $a \in A$, а $b \notin A$. По предположению индукции $d[a]$ уже равно кратчайшему расстоянию до a , и при обработке a ребро (a, b) было прорелаксировано. Поэтому

$$d[b] \leq d[a] + w(a, b).$$

Так как все веса неотрицательны, расстояние до b вдоль рассматриваемого пути не больше полной длины пути до v . Следовательно, если путь до v был бы короче $d[v]$, то получили бы $d[b] < d[v]$. Но v выбрана как необработанная вершина с минимальной оценкой d , противоречие. Значит $d[v]$ уже оптимально. $\square$

18.3 Сложность

Пусть $n = |V|$, $m = |E|$. Общая схема оценки:

$$T = n \cdot T_{\text{extractMin}} + m \cdot T_{\text{relax}}.$$

Реализация	Сложность
Без кучи, поиск минимума линейным просмотром	$O(n^2 + m)$
Бинарная куча	$O((n + m) \log n)$, часто пишут $O(m \log n)$ при $m \geq n - 1$
Фибоначчиева куча	$O(m + n \log n)$ амортизированно
Pairing heap	хороша на практике; обычно используют как простую альтернативу более сложным кучам

Для плотных графов вариант $O(n^2 + m)$ может быть не хуже реализации с кучей. Для разреженных графов обычно используют бинарную кучу или другую приоритетную очередь.

18.4 Восстановление пути

Если при каждой успешной релаксации записывать $\text{parent}[x] = v$, то кратчайший путь до вершины t восстанавливается обратным проходом:

```
1 path = []
2 while t != -1:
3     path.push_back(t)
4     t = parent[t]
5 reverse(path)
```

Если после алгоритма $d[t] = +\infty$, вершина t недостижима из s , и пути нет.

18.5 Ограничения и замечания

Ключевое условие корректности Дейкстры — неотрицательность весов. При отрицательных рёбрах вершина, уже извлечённая как минимальная, может позже получить лучшее расстояние, поэтому обычное доказательство ломается. Для графов с отрицательными весами используют другие алгоритмы, например Беллмана–Форда или Флойда–Уоршелла.

Если нужно найти кратчайший путь только до одной вершины t , алгоритм можно остановить сразу после извлечения t из очереди: в этот момент расстояние до t уже окончательно по теореме корректности.

19 Графы. Минимальные остовные деревья. Алгоритм Прима. Корректность и сложность алгоритма.

19.1 Минимальное остовное дерево

Остовное дерево

Пусть дан связный неориентированный граф $G = (V, E)$. Остовное дерево — подмножество рёбер $T \subseteq E$, которое соединяет все вершины и не содержит циклов. В любом остовном дереве ровно $|V| - 1$ ребро.

Если у каждого ребра e есть вес $w(e)$, вес остова равен

$$w(T) = \sum_{e \in T} w(e).$$

Задача MST (*minimum spanning tree*) — найти остовное дерево минимального веса. Если граф несвязен, вместо остовного дерева говорят о минимальном остовном лесу.

Лемма о разрезе

Пусть вершины графа разбиты на два непустых множества

$$V = A \sqcup B.$$

Пусть e — ребро минимального веса среди всех рёбер, пересекающих разрез (A, B) . Тогда существует минимальное остовное дерево, содержащее e .

Доказательство. Возьмём произвольное минимальное остовное дерево T . Если $e \in T$, доказывать нечего. Иначе добавим e к T . В дереве появится ровно один цикл. Так как e пересекает разрез, цикл должен пересечь этот разрез ещё хотя бы одним ребром f . По выбору e имеем $w(e) \leq w(f)$. Удалим f из цикла. Получим остовное дерево

$$T' = T - f + e,$$

причём $w(T') \leq w(T)$. Так как T было минимальным, T' тоже является MST и содержит e . $\square$

19.2 Алгоритм Прима

Алгоритм Прима строит MST, постепенно расширяя множество уже присоединённых вершин A . На каждом шаге выбирается самое лёгкое ребро, которое выходит из A в $V \setminus A$.

```

1  Prim(start):
2      for v in V:
3          key[v] = INF
4          parent[v] = -1
5          used[v] = false
6      key[start] = 0
7
8      repeat n times:
9          v = unused vertex with minimum key[v]
10         used[v] = true
11         for edge (v, x, w) in g[v]:
12             if not used[x] and w < key[x]:
13                 key[x] = w
14                 parent[x] = v

```

Здесь $\text{key}[v]$ — вес минимального известного ребра, соединяющего v с уже построенной частью остова. Рёбра $(\text{parent}[v], v)$ для всех $v \neq \text{start}$ образуют найденный остов.

Корректность алгоритма Прима

Если граф связан, алгоритм Прима строит минимальное остовное дерево.

Доказательство. Будем поддерживать инвариант: уже выбранные алгоритмом рёбра содержатся в некотором минимальном остовном дереве. Изначально выбранных рёбер нет, инвариант очевиден.

Пусть алгоритм уже построил множество вершин A и выбирает минимальное ребро e , выходящее из A в $V \setminus A$. Рассмотрим разрез $(A, V \setminus A)$. Ребро e является минимальным на этом разрезе, поэтому по лемме о разрезе существует MST, содержащий e . Более точно, если учитывать уже выбранные рёбра, они целиком лежат внутри A , поэтому не пересекают текущий разрез, и обменный аргумент из леммы не ломает уже построенную часть.

Значит после добавления e инвариант сохраняется. После $|V| - 1$ добавлений выбранные рёбра образуют остовное дерево и при этом содержатся в некотором MST, следовательно, сами являются минимальным остовом. $\square$

19.3 Сложность

Без кучи минимум среди неиспользованных вершин ищется линейным просмотром:

$$O(n^2 + m).$$

Для плотных графов это часто удобная реализация.

Если хранить значения `key` в приоритетной очереди, то алгоритм работает как Дейкстра с другой формулой обновления: вместо $d[v] + w(v, x)$ используется просто вес последнего ребра $w(v, x)$. С бинарной кучей:

$$O((n + m) \log n).$$

С более сильной кучей и настоящим `decreaseKey` можно получить

$$O(m + n \log n).$$

19.4 Алгоритм Краскала

Алгоритм Краскала строит минимальный остов «снизу вверх»: поддерживается лес выбранных рёбер, и рёбра рассматриваются в порядке неубывания веса. Очередное ребро добавляется только если оно соединяет две разные компоненты текущего леса. Для проверки компонент используется DSU.

```
1 Kruskal():
2     sort edges by weight
3     make_set(v) for every vertex v
4     mst = []
5     for edge (u, v, w) in edges:
6         if find(u) != find(v):
7             mst.push(edge)
8             union(u, v)
9     return mst
```

Инвариант алгоритма: множество уже выбранных рёбер ациклично и содержится в некотором MST. Когда рассматривается ребро $e = (u, v)$, соединяющее разные компоненты леса, возьмём разрез, одна сторона которого содержит компоненту u , а другая — остальные вершины. Все более лёгкие рёбра этого разреза уже были рассмотрены; если они не были добавлены, то соединяли вершины внутри одной компоненты текущего леса. Следовательно, e является минимальным допустимым ребром для некоторого разреза, и по лемме о разрезе его можно добавить, не потеряв существование MST, содержащего выбранные рёбра. Рёбра, соединяющие вершины одной компоненты, отбрасываются, потому что они создают цикл.

После выбора $|V| - 1$ рёбер в связном графе получается остовное дерево. По инварианту оно содержится в некотором MST, значит само является минимальным остовным деревом. Если граф несвязен, алгоритм строит минимальный остовный лес.

Сложность определяется сортировкой рёбер и операциями DSU:

$$O(m \log m + m\alpha(n)) = O(m \log m),$$

где $\alpha(n)$ — обратная функция Аккермана, возникающая в DSU с эвристиками сжатия путей и объединения по рангу или размеру. Так как $m \leq n^2$, часто также пишут $O(m \log n)$.

Прим обычно удобен на плотных графах и когда граф задан списками смежности с быстрым обновлением ключей. Краскал часто проще реализовать и хорошо подходит для разреженных графов, если все рёбра уже доступны одним списком.

20 Графы. Задача поиска кратчайших расстояний в графе. Алгоритм Беллмана–Форда. Поиск циклов отрицательного веса.

20.1 Задача и отличие от Дейкстры

Алгоритм Беллмана–Форда решает задачу SSSP в графе с произвольными весами рёбер. В отличие от Дейкстры, он допускает отрицательные рёбра. Если из стартовой вершины достижим цикл отрицательного веса, то кратчайшие расстояния до некоторых вершин не определены: по циклу можно пройти много раз и уменьшать длину пути без ограничения.

20.2 Динамическая идея

Обозначим через $d_k[v]$ минимальный вес пути из s в v , использующего не более k рёбер. Тогда

$$d_0[s] = 0, \quad d_0[v] = +\infty \text{ при } v \neq s,$$

а переход имеет вид

$$d_{k+1}[v] = \min \left(d_k[v], \min_{(u,v) \in E} (d_k[u] + w(u, v)) \right).$$

Внутренний минимум берётся по всем рёбрам, входящим в вершину v ; первое слагаемое $d_k[v]$ соответствует случаю, когда новый путь не использует $(k+1)$ -е ребро. Если отрицательных циклов, достижимых из s , нет, то кратчайший путь можно выбрать простым, а значит он содержит не более $n-1$ рёбер. Поэтому достаточно $n-1$ фаз.

20.3 Алгоритм

```

1 BellmanFord(s):
2   for v in V:
3       d[v] = INF
4       parent[v] = -1
5   d[s] = 0
6
7   for i = 1; i <= n - 1; i++:
8       for edge (u, v, w) in edges:
9           if d[u] != INF and d[v] > d[u] + w:
10                d[v] = d[u] + w
11                parent[v] = u

```

Корректность Беллмана–Форда

После k -й фазы алгоритм уже учёл все пути из s в v , использующие не более k рёбер. Каждое конечное значение $d[v]$ при этом является весом некоторого пути из s в v . После $n - 1$ фаз, если достижимых отрицательных циклов нет, все расстояния найдены корректно.

Доказательство. Докажем первую часть по индукции по k . При $k = 0$ известен только путь длины 0 из s в себя, и инициализация верна.

Пусть утверждение верно после k фаз. Рассмотрим путь из не более чем $k + 1$ рёбер, оканчивающийся в v . Если в нём не более k рёбер, оценка уже была учтена раньше. Иначе последнее ребро имеет вид (u, v) , а префикс пути до u содержит не более k рёбер. По предположению индукции после k фаз путь до u уже учтён, а на следующей фазе ребро (u, v) релаксируется. Значит путь до v тоже будет учтён.

Каждое конечное значение $d[v]$ возникает либо из начального $d[s] = 0$, либо после релаксации некоторого ребра (u, v) , то есть соответствует реальному пути до u , дополненному этим ребром. Поэтому алгоритм не может получить значение меньше истинного кратчайшего расстояния.

Если отрицательных циклов нет, кратчайший путь можно сделать простым: повтор вершины образует цикл, удаление которого не увеличивает вес пути. Простой путь содержит не более $n - 1$ рёбер, поэтому после $n - 1$ фаз все кратчайшие расстояния найдены. $\square$

20.4 Сложность

На каждой фазе просматриваются все m рёбер, фаз $n - 1$. Поэтому

$$T(n, m) = O(nm).$$

Память равна $O(n)$ для массива расстояний и родителей, если список рёбер уже хранится во входном графе.

20.5 Поиск отрицательного цикла

Чтобы проверить наличие достижимого отрицательного цикла, выполняют ещё одну, n -ю, фазу релаксаций.

Критерий отрицательного цикла

После $n - 1$ фаз алгоритма Беллмана–Форда достижимый из s отрицательный цикл существует тогда и только тогда, когда на n -й фазе возможна хотя бы одна успешная релаксация.

Доказательство. Если после $n - 1$ фаз возможна успешная релаксация, то отсутствие достижимых отрицательных циклов невозможно: по теореме корректности в таком случае после $n - 1$ фаз все расстояния уже были бы кратчайшими, и ни одно ребро не могло бы их улучшить. Значит существует достижимый из s отрицательный цикл.

Обратно, пусть из s достижим отрицательный цикл

$$v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k = v_0.$$

После $n - 1$ фаз все вершины этого цикла имеют конечные оценки. Если бы ни одно ребро цикла нельзя было прорелаксировать, то для каждого ребра (v_i, v_{i+1}) выполнялось бы

$$d[v_{i+1}] \leq d[v_i] + w(v_i, v_{i+1}).$$

Просуммировав эти неравенства по циклу, получили бы

$$0 \leq \sum_i w(v_i, v_{i+1}),$$

что противоречит отрицательному весу цикла. Следовательно, хотя бы одна релаксация на n -й фазе возможна. $\square$

20.6 Восстановление отрицательного цикла

При каждой успешной релаксации сохраняем $\text{parent}[v] = u$. Если на n -й фазе релаксировалась вершина x , то для попадания внутрь цикла нужно n раз пройти по родителям:

```
1 y = x
2 for i = 1; i <= n; i++:
3     y = parent[y]
```

После этого y гарантированно лежит на отрицательном цикле. Чтобы выписать цикл, идём по родителям от y , пока снова не вернёмся в y .

```
1 cycle = []
2 cur = y
3 do:
4     cycle.push_back(cur)
5     cur = parent[cur]
6 while cur != y
7 cycle.push_back(y)
8 reverse(cycle)
```

Эта процедура работает за $O(n)$ после основного алгоритма.

20.7 Практические оптимизации

Если на очередной фазе не произошло ни одной релаксации, алгоритм можно остановить раньше: дальше массив расстояний уже не изменится. Такая оптимизация не меняет худшую оценку $O(nm)$, но часто существенно ускоряет работу.

Существует очередь-версия, часто называемая SPFA: в очередь кладутся только вершины, чьи расстояния недавно улучшились. Она нередко быстра на практических тестах, но в худшем случае также может работать за $O(nm)$.